
RAPPORT DE PROJET

NeuralSpeech

RÉSUMÉ – Dans le cadre de notre première année du cycle ingénieur à l'ECE, Ecole Centrale d'Electronique, notre équipe composée de Guy-Charbel KAIROUZ, Thaïs LECLAIRE, Thomas LEROY, Lucie DAIX devait relever le défi de réaliser une reconnaissance vocale basée sur une IA embarquée en seulement 5 semaines et avec la contrainte d'utiliser le matériel électronique imposé.

L'objectif était de mettre à profit les connaissances et les compétences acquises lors du module calcul embarqué au travers d'un cahier des charges ayant pour finalité la création d'une reconnaissance vocale.

Pour accomplir ce projet, il était nécessaire de maîtriser la configuration de l'ADC de la carte Arduino DUE, l'utilisation d'algorithmes de traitement du signal et la conception de réseau de neurones.

Ce rapport est la retranscription de chaque étape du développement de NeuralSpeech, de la conception à la validation de chaque test. Il est le fruit d'un travail rudement mené par une équipe passionnée par l'électronique qui a saisi l'opportunité d'exploiter cet intérêt commun pour soumettre l'ébauche d'un projet innovant.

Guy-Charbel KAIROUZ

Thaïs LECLAIRE

Thomas LEROY

Lucie DAIX

Nous attestons que ce travail est original,
qu'il est le fruit d'un travail commun au binôme et qu'il a été rédigé de manière autonome.

Paris, le 12/05/2025

Table des matières

1 Objectifs	3
2 Glossaire	3
2.1 Termes	3
2.2 Acronymes	3
3 L'équipe	4
3.1 Présentation de l'équipe	4
3.2 Organisation de l'équipe	4
3.3 Diagramme de Gantt	4
4 Contexte et problématique	5
4.1 Contexte	5
4.2 Problématique	5
4.3 Spécifications techniques	5
5 Conception	5
5.1 Architecture fonctionnelle	5
5.2 Architecture matérielle	6
5.3 Architecture logicielle	7
6 Développement	8
6.1 Acquisition du signal	8
6.1.1 Numériser le signal audio	9
6.1.2 Conditionnement du signal	10
6.1.3 Caractériser le timbre vocal	12
6.1.4 Subsample le signal	13
6.1.5 MFCC et DCT	14
6.2 Traitement des données	15
6.2.1 Introduction aux réseaux neuronaux	15
6.2.2 Echanges des données inter-plateformes	17
6.2.3 Classification des enregistrements	18
6.2.4 Élargissement de la classification	21
7 Tests et validation	23
7.1 Acquisition du signal	23
7.1.1 Numériser le signal audio	23
7.1.2 Conditionnement du signal	25
7.1.3 Subsample le signal	27
7.1.4 Écouter et valider l'enregistrement	28
7.1.5 Caractériser le timbre vocal	29
7.2 Traitement des données	32
7.2.1 Entraînement du réseau neuronal de test	32
7.2.2 Entraînement du réseau neuronal à deux sorties possibles	32
7.2.3 Fonctionnalité avancée : entraînement du réseau neuronal à quatre sorties	34
8 Bilan	36
8.1 État d'avancement	36
8.2 Pertinence de la solution technique	37
8.3 Bilan sur le travail d'équipe	37
9 Sources	39

1 Objectifs

Ayant un sentiment commun pour l'électronique, notre groupe composé de Lucie DAIX, Guy-Charbel KAIROUZ, Thaïs LECLAIRE et Thomas LEROY, a saisi l'opportunité d'exploiter cet intérêt commun pour soumettre un projet personnel innovant à notre responsable Madame GULDNER.

Le but de ce projet est d'acquérir des compétences techniques et informatiques en concevant, fabriquant et testant une reconnaissance vocale en travaillant en groupe et en respectant les exigences techniques préalablement transmises.

2 Glossaire

2.1 Termes

Termes	Définitions
Arduino	Plateforme de prototypage permettant de créer des objets électroniques interactifs à partir de cartes électroniques
Audacity	Logiciel d'enregistrement et d'édition de son numériques sous différents formats
PuTTY	Emulateur de terminal

2.2 Acronymes

Acronyme	Signification	Explication
ET	Exigence technique	Exigences techniques du cahier des charges.
ADC	Analog to Digital Converter	Dispositif dont la fonction est de traduire une grandeur analogique en une valeur numérique.
DAC	Digital to Analog Converter	Dispositif dont la fonction est de traduire une grandeur numérique en une valeur analogique.
FFT	Fast Fourier Transform	Cet algorithme utilisé pour transformer des données discrètes du domaine temporel dans le domaine fréquentiel.
MFCC	Mel-Frequency Cepstral Coefficients	Les MFCC ou Mel-Frequency Cepstral Coefficients sont des coefficients cepstraux calculés par une transformée en cosinus discrète appliquée au spectre de puissance d'un signal.
DCT	Discrete Cosine Transform	Algorithme permettant de regrouper l'énergie par les coefficients basse fréquence.

3 L'équipe

3.1 Présentation de l'équipe

Notre équipe est composée de :

- Lucie Daix, 20 ans, reconnue comme la femme la plus drôle et la plus avenante du groupe. Sans elle, une monotonie s'installerait. Et sans ses clés, elle ne rentrerait pas chez elle. Spoiler Alert : elle n'est pas rentrée chez elle.
- Guy-Charbel Kairouz, également appelé le RAGEUX, est d'ores et déjà destiné à devenir l'homme le plus en retard du monde d'ici l'année prochaine. Pourquoi ? La station Vélib était pleine.
- Thaïs Leclaire, femme la plus chiante du monde, a essayé de l'être un peu moins le temps du projet. Après coup, les résultats sont formels : elle a échoué.
- Thomas Leroy a eu la chance de découvrir la ville de Montpellier pendant ce projet, et accessoirement de faire le réseau neuronal qui fonctionne.

3.2 Organisation de l'équipe

La travail a été réparti de manière équitable. Nous étions très solidaires dans la difficulté.

La rédaction du rapport a été réalisée par l'ensemble des membres du groupe au fur et à mesure. L'acquisition du signal sonore a été une partie dirigée essentiellement par Lucie, Thaïs et Guy-Charbel. L'entraînement du réseau de neurone a été réalisé en majeure partie par Thomas et Guy-Chabel. Enfin, l'ensemble du groupe a traité la validation et les tests des différents modules grâce à l'expérience acquise du travail en amont.

3.3 Diagramme de Gantt

Dans le but de visualiser l'ordonnancement des tâches, les dépendances entre elles, les durées et les échéances de notre projet, nous avons utilisé un diagramme de GANTT.

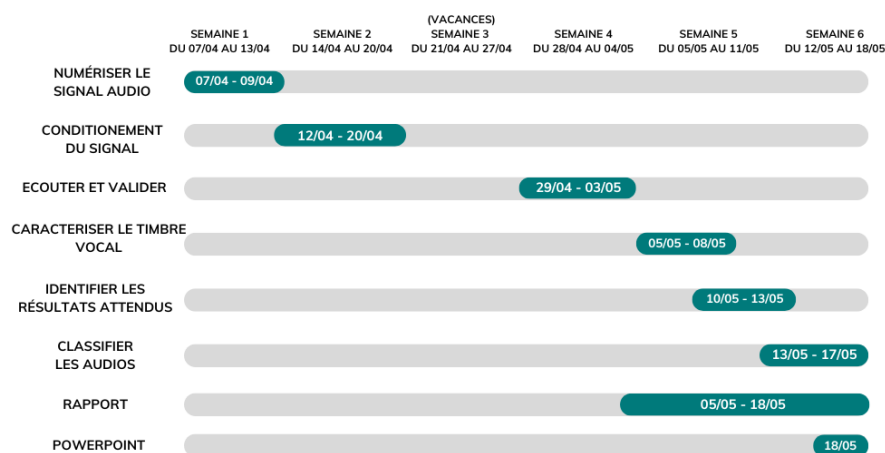


FIGURE 1 – Diagramme de Gantt

4 Contexte et problématique

4.1 Contexte

Dans le cadre de notre première année du cycle ingénieur (3ème année post bac) à l'ECE, il nous est proposé un projet de six semaines nous permettant de mettre en pratique nos connaissances et compétences acquises durant notre semestre en Calcul Embarqué au travers d'un cahier des charges ayant pour finalité la création d'une reconnaissance vocale intelligente.

4.2 Problématique

La compréhension de l'acquisition du signal et de l'entraînement neuronal au cœur de notre projet a été nécessaire. De multitudes réunions de travail en présentiel et en distanciel ont été indispensables pour penser, concevoir et tester le projet avec la rédaction du rapport au fur et à mesure.

Nous verrons donc comment parvenir à concevoir et réaliser le projet Neural Speech compte tenu des exigences techniques et limitations matérielles qui nous sont imposées.

Bien entendu, les cours et les DS se poursuivent en parallèle.

4.3 Spécifications techniques

La réalisation du projet doit répondre à des spécifications techniques telles que l'utilisation de la carte Arduino DUE configurable sur le logiciel Arduino IDE.

5 Conception

5.1 Architecture fonctionnelle

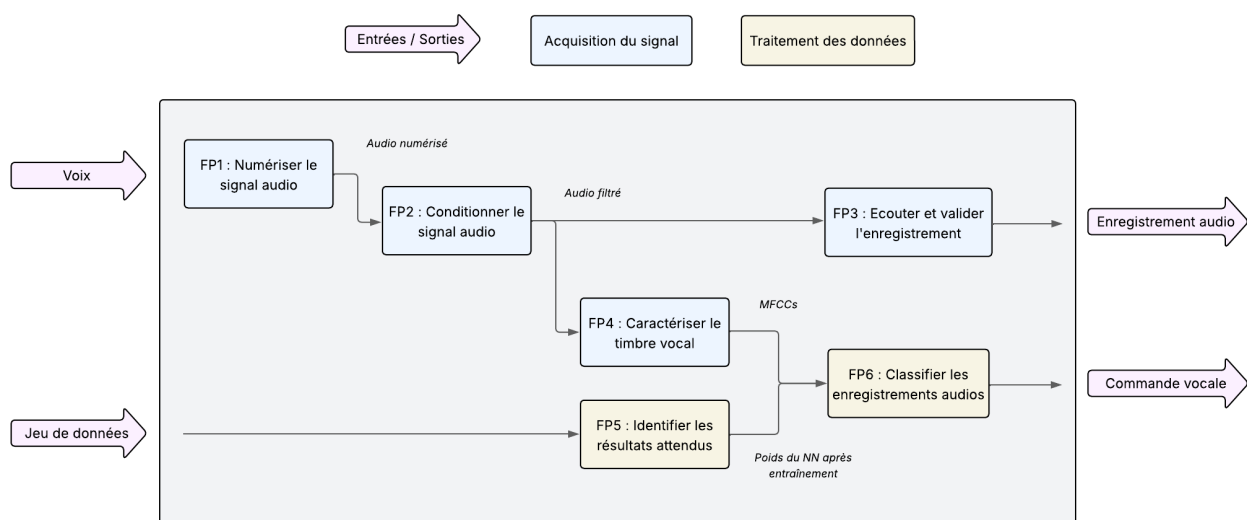


FIGURE 2 – Diagramme fonctionnel

5.2 Architecture matérielle

Pour développer chaque étape de notre circuit, il était nécessaire de se renseigner sur les différents composants électroniques qui le composent.

Pour réaliser notre reconnaissance vocale, nous nous sommes munis de l'ensemble des composants électroniques qui figurent dans le tableau ci-dessous :

Composant	Fonction
1 breadboard	Support principal
1 Arduino DUE	Support principal
1 Bouton poussoir	Commencer l'enregistrement
1 LED rouge	Visualiser la seconde d'enregistrement
4 LED (rouge, jaune, bleue, verte)	Une LED correspond à un mot. Si le mot est reconnu, la LED correspondante s'allume
1 Micro MAX9814	Capter le signal sonore
Câbles (mâles-femelles)	Connecter chaque composant électronique

Voici l'architecture matérielle en platine d'essai et en vue schématique :

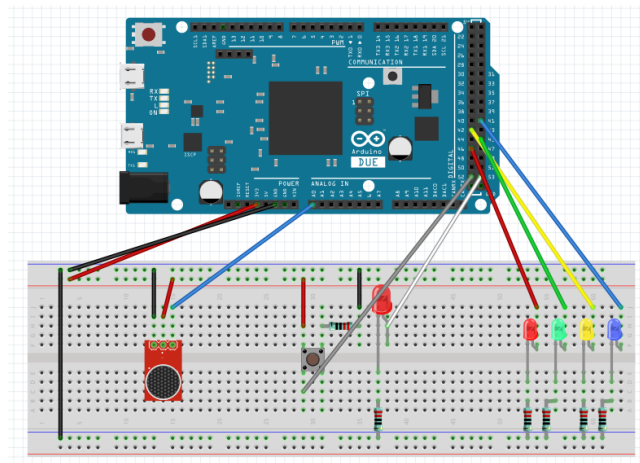


FIGURE 3 – architecture matérielle en vue platine d'essai

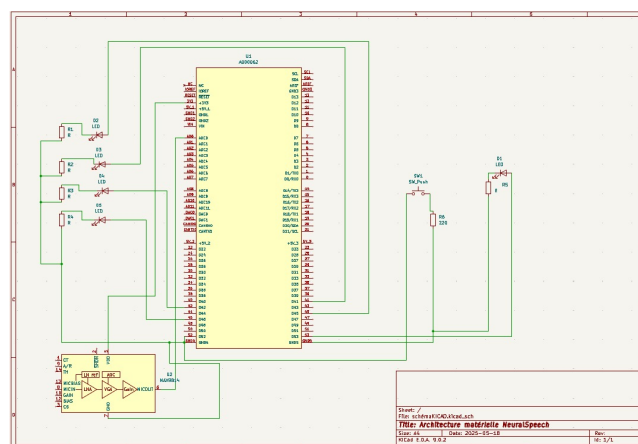


FIGURE 4 – architecture matérielle en vue schématique

5.3 Architecture logicielle

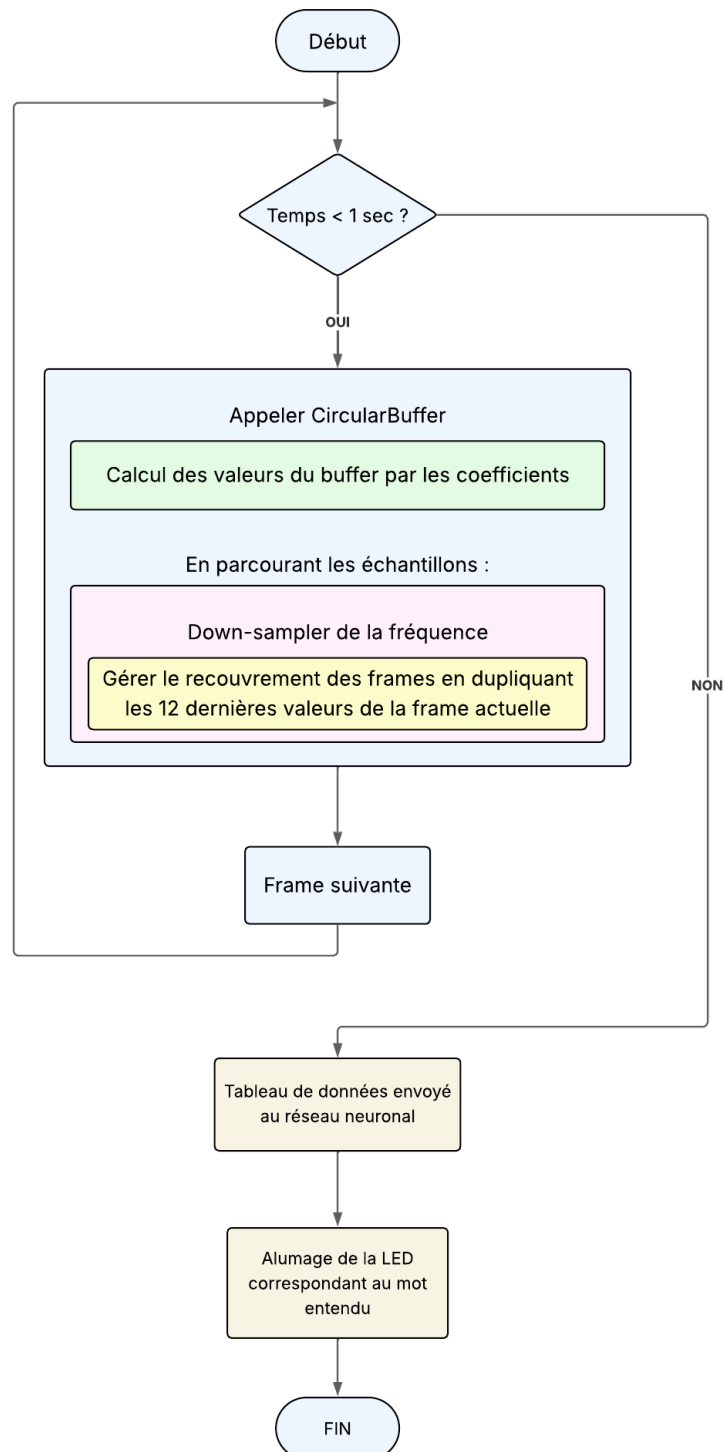


FIGURE 5 – Algorithme du fil d'exécution d'un tour de boucle de loop

6 Développement

Le projet complet du Neural Speech peut se répartir en deux parties principales.

Dans un premier temps, nous traiterons la partie concernant l'acquisition du signal audio.
Puis dans un second temps, nous traiterons la partie entraînement du réseau de neurones.

6.1 Acquisition du signal

Voici l'algorithme général, qui permet de visualiser clairement les évènements qui se déroulent dans le code :

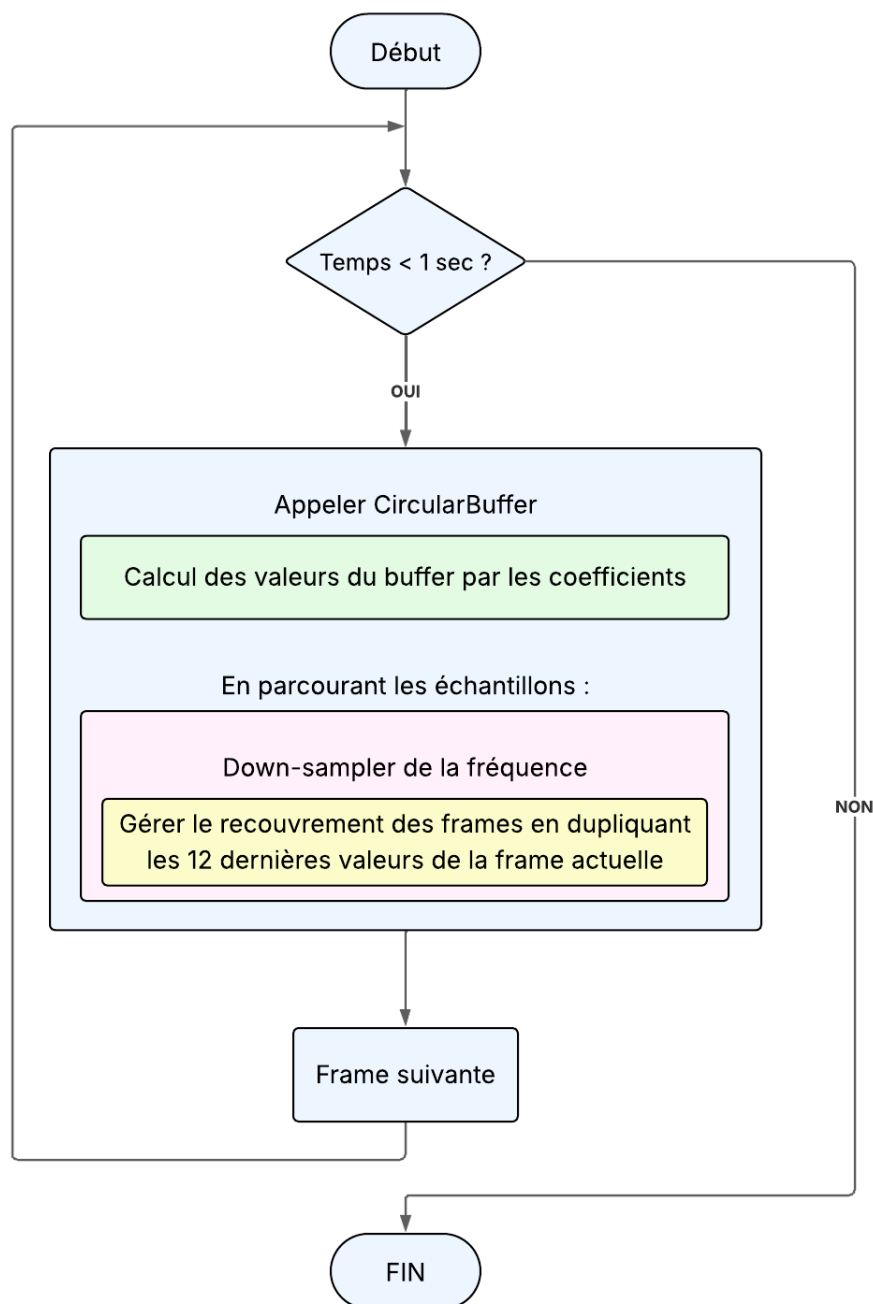


FIGURE 6 – Algorithme général de l'acquisition du signal

6.1.1 Numériser le signal audio

La numérisation d'un signal audio commence par une phase d'échantillonnage. L'échantillonnage est le processus consistant à prélever des valeurs (échantillons) d'un signal continu à intervalles réguliers pour les convertir en un signal numérique. Cette conversion est essentielle pour que les ordinateurs et autres dispositifs numériques puissent traiter et stocker des signaux.

Dans notre cas, nous utilisons l'ADC (Analog-to-Digital Converter) de notre Arduino DUE, configuré avec une précision de 12 bits. Ce composant électronique assure la conversion d'une tension analogique provenant de notre micro MAX9814 en une valeur numérique (entre 0 et 4095) que le microcontrôleur de l'Arduino peut traiter.

Nous venons de comprendre l'intérêt de procéder à un échantillonnage pour numériser un signal audio. Maintenant voyons comment nous l'avons mis en place dans l'intérêt de notre projet.

L'échantillonnage est mis en place informatiquement. C'est pourquoi nous l'avons entièrement codé.

Tout d'abord, nous avons configuré un minuteur appelé Timer. Il permet de générer les interruptions à un intervalle de temps précis. La valeur de cet intervalle de temps, appelée RC, est à calculer. En effet, celle-ci représente la valeur maximum que le minuteur va atteindre avant d'activer l'interruption.

Nous utilisons la formule suivante pour déterminer cette valeur :

$$RC = \frac{\frac{MCK}{Pr}}{F}$$

Nous savons que la fréquence d'horloge principale (MCK) de l'Arduino DUE est de 84 MHz. Nous utilisons un prescaler (Pr) à 2. Et F correspond à fréquence d'échantillonnage souhaitée, ici 32 kHz.

Le calcul effectué, nous obtenons la valeur de RC soit 1312.5.

Reprenons la configuration de notre Timer. Tout d'abord on active le périphérique Timer, puis nous paramétrons le contrôle du Timer en utilisant une interruption sur RC Compare. Plusieurs registres nous sont nécessaires. Premièrement, on configure l'horloge qui servira à compter jusqu'à la valeur voulue. Pour ce faire, il faut mettre à un les bits TC CMR WAVE, TC CMR TCCLKS TIMER CLOCK1 et TC CMR CPCTRIG. Cela permet respectivement de choisir la forme du signal de la clock (ici des signaux PWM), sa fréquence et son mode de fonctionnement, ici le mode RC compare.

Il faut ensuite configurer le registre TC RC à la bonne fréquence d'échantillonnage, puis mettre à un le bit TC IER CPCS du registre CPCS pour activer l'interruption sur RC Compare. Enfin, le bit TC0 IRQn active l'interruption dans le NVIC. Il ne reste alors plus qu'à activer le timer à l'aide des bits CLKEN et SWTRG dans le registre TC CCR. Le timer va alors s'incrémenter jusqu'à la valeur définie par RC, puis déclencher la fonction d'interruption à ce moment-là. Celle-ci restera active tant que le bit TC SR restera à 1.

Pour activer le timer TC0, on active les bits TC CCR CLKEN et TC CCR SWTRG du registre TC CCR afin de démarrer la clock du timer et d'enclencher le décompte.

Afin d'activer cette fois l'interruption du timer TC0, on met à 1 le bit TC IER CPCS du registre TC IER. C'est une interruption par RC Compare dont le temps entre chaque interruption est stocké dans le registre TC RC. Le timer compte et compare sa valeur avec celle stockée dans RC. A l'égalité des deux, l'interruption est déclenchée et le compteur recommence. On récupère cette interruption dans le TC0 Handler.

Le drapeau d'interruption TC SR est mis à 1 lorsque l'interruption timer a lieu. Si celui-ci n'est pas clear, alors le programme reste bloqué dans une interruption infinie. Il est donc nécessaire de clear ce

drapeau, ce qui est faisable en lisant simplement le registre en question.

Par la suite, on configure l'ADC. Pour ce faire, il suffit de changer les paramètres du registre ADC MR : le bit ADC MR PRESCAL à 3 permet de modifier la valeur du prescaler utilisé pour diviser l'horloge de l'ADC, le bit ADC MR STARTUP SUT64 afin d'activer le temps de démarrage de 64 périodes, le bit ADC MR TRACKTIM(16) pour définir 16 périodes entre chaque basculement, et enfin le bit ADC MR SETTLING AST3 pour fixer le temps de stabilisation de 3 périodes. Le reste des paramètres sont définis par défaut aux valeurs souhaitées ; nous laissons alors les autres bits à zéro.

Pour sélectionner l'entrée de l'ADC à convertir, il faut utiliser le registre ADC CHER, en mettant à 1 le bit ADC CHER CH7 pour activer l'entrée A0 à laquelle le micro est branché.

Enfin, le lancement de la conversion analogique s'effectue lors de l'interruption du timer en mettant à 1 le bit ADC CR START du registre ADC CR. Pour exécuter un code lorsque lorsque le drapeau d'interruption de fin de conversion est mis à 1, il suffit d'inclure une condition dans la loop principale du programme permettant de vérifier l'état du registre ADC ISR et du bit ADC ISR DRDY (qui passe à 1 lorsque la conversion est prête – c'est le bit DataReady). Puisque cette partie est gérée dans la loop, la fonction Handler de l'interruption de l'ADC n'est donc pas nécessaire.

6.1.2 Conditionnement du signal

Dans le cadre du projet, l'implémentation d'un filtre passe-bas était essentielle afin d'éliminer les fréquences supérieures à 4 000 Hz, tout en assurant une atténuation d'au moins 30 dB. Une contrainte supplémentaire imposait que le temps de filtrage par échantillon ne dépasse pas 31 μ s, garantissant une exécution rapide adaptée aux exigences du système.

Nous avons le choix entre utiliser un filtre RII ou bien un filtre RIF. Notre choix s'est porté sur le filtre RIF pour les raisons suivantes :

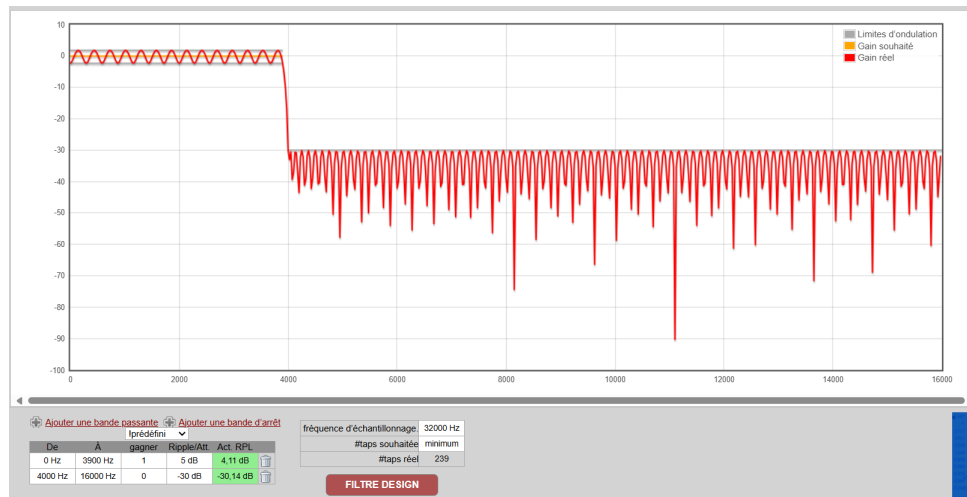
Les RIF peuvent être facilement conçus pour avoir une **phase linéaire**. Cela signifie qu'aucune distorsion de phase n'est introduite dans le signal à filtrer, car toutes les fréquences sont décalées dans le temps de la même quantité – ce qui maintient leurs relations harmoniques relatives (c'est-à-dire un retard de groupe et de phase constant). Ce n'est certainement pas le cas avec les filtres RII, qui ont une caractéristique de phase non linéaire.

De plus, comme les filtres RIF n'utilisent pas les valeurs de sortie précédentes pour calculer leur sortie actuelle, c'est-à-dire qu'ils n'ont pas de rétroaction, ils ne peuvent **jamais devenir instables** pour tout type de signal d'entrée, ce qui leur donne un avantage distinct sur les filtres RII.

Cependant, le filtre RIF demande plus de coefficients que le filtre RII donc il prend plus de temps à filtrer un signal.

Pour concevoir un filtre efficace, nous avons opté pour une approche qui privilégie la linéarité du son afin d'éviter toute distorsion indésirable lors du filtrage. Pour optimiser le temps de traitement, nous avons réduit le nombre de coefficients du filtre RIF, trouvant ainsi un équilibre entre qualité sonore et rapidité d'exécution.

Pour concevoir un filtre optimal, une simulation numérique a été réalisée sur le site : t-filter.engineerjs.com

**FIGURE 7** – Simulation filtre RIF

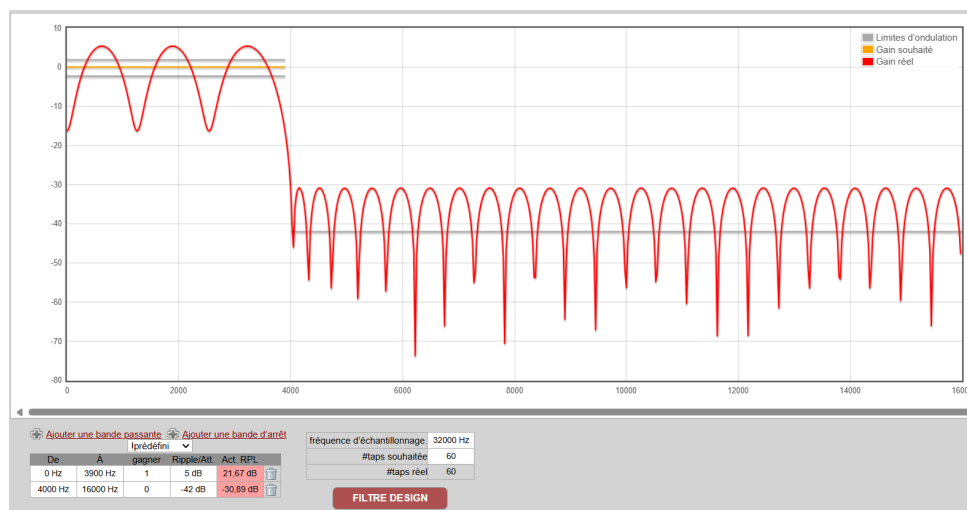
Pour une précision maximale, cette simulation a généré 239 coefficients du filtre RIF. Une fois intégrés au code fourni sur Github ([Lien du code GitHub](#)), des tests de performance ont été effectués en utilisant un compteur de temps.

Cependant, les premiers résultats ont révélé une latence bien supérieure aux attentes : chaque filtrage d'échantillon prenait environ 90 μ s, soit trois fois le délai maximal autorisé. Cette surcharge temporelle était directement liée au nombre élevé de coefficients du filtre, entraînant une quantité excessive de calculs à chaque itération.

Afin d'optimiser ce temps de filtrage, la seule solution fut de réduire le nombre de coefficients du filtre RIF, diminuant ainsi la charge de calcul.

Après optimisation, nous avons pu ramener le temps de traitement en dessous des 31 μ s exigés, garantissant ainsi une exécution conforme aux spécifications techniques du projet.

Voici notre configuration finale :

**FIGURE 8** – Simulation filtre RIF, 60 coefficients

Cette adaptation a permis d'améliorer l'efficacité du système tout en conservant une qualité de filtrage satisfaisante.

6.1.3 Caractériser le timbre vocal

Pour assurer un enregistrement sonore sans perte d'information, il était essentiel d'introduire une superposition entre les frames afin de préserver la continuité du signal et éviter les discontinuités qui pourraient altérer l'analyse.

Nous avons décidé de mettre en place un chevauchement de 12 échantillons sur 256 (PRECISION = 12), un choix arbitraire mais réfléchi. Un chevauchement trop important aurait engendré un nombre excessif de frames, complexifiant le traitement du signal et augmentant inutilement la charge de calcul. Il s'agissait donc de trouver un compromis entre la qualité du signal et l'efficacité du traitement. De plus, par la suite, nous aurons besoin de down-sampler le signal par 4 (afin d'atteindre une fréquence de 8 kHz). Il était donc pertinent de prendre un multiple de 4.

Concrètement, sans recouvrement, le nombre de frames obtenues sur une seconde de signal est de 126. En intégrant un recouvrement de 12 échantillons, ce nombre passe à 132, offrant une meilleure prise en compte des transitions entre frames et garantissant une continuité dans l'analyse du signal. Ce recouvrement permet d'améliorer la précision de la représentation spectrale et d'optimiser la reconnaissance vocale sans surcharge inutile.

Voici l'algorithme du recouvrement :

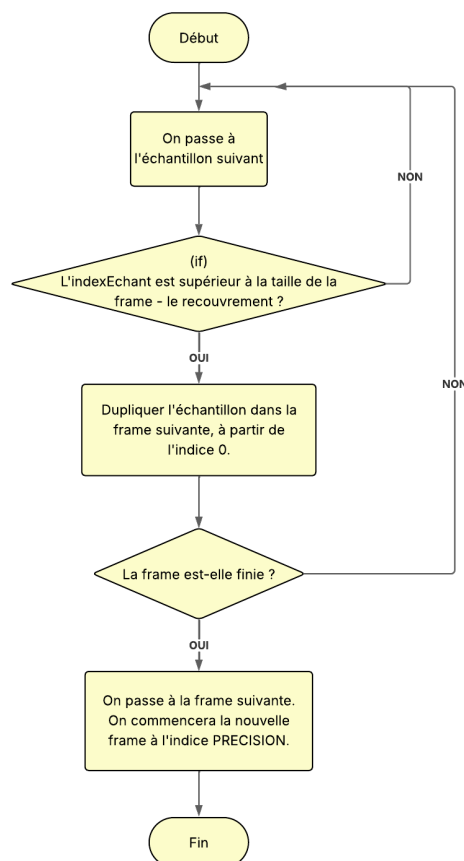


FIGURE 9 – Algorithme de recouvrement de frames

6.1.4 Subsampler le signal

Après le filtrage des échantillons et la gestion du recouvrement à 32 kHz, il a été nécessaire de réduire la fréquence d'un facteur 4, procéder à un down-sampling, afin d'atteindre une fréquence de 8 kHz.

Initialement, nous avons envisagé de sélectionner un échantillon sur quatre, mais cette approche aurait entraîné une perte significative de précision. Afin de préserver la qualité du signal, nous avons opté pour une méthode plus rigoureuse : calculer la moyenne de chaque groupe de quatre échantillons, permettant ainsi d'obtenir une représentation plus fidèle et homogène du signal traité.

Pour cela, nous constituons un tableau temporaire de 4 valeurs. En parcourant nos 256 échantillons sur 132 frames, nous ajoutons une valeur à ce tableau à chaque itération. Une fois le tableau rempli (toutes les 4 itérations), nous en calculons la moyenne. Cette moyenne est ensuite stockée dans un tableau final, qui contiendra au total 64 échantillons moyennés pour chacune des 132 frames.

Afin d'améliorer l'efficacité des boucles de calcul, nous avons intégré le recouvrement des frames (vu dans le 6.3) directement dans le processus de réduction de fréquence. Cette approche permet de traiter simultanément les deux opérations plutôt que de les exécuter séparément, réduisant ainsi le nombre d'itérations nécessaires.

Pour mieux visualiser nos explications, voici l'algorithme détaillé du subsampling.

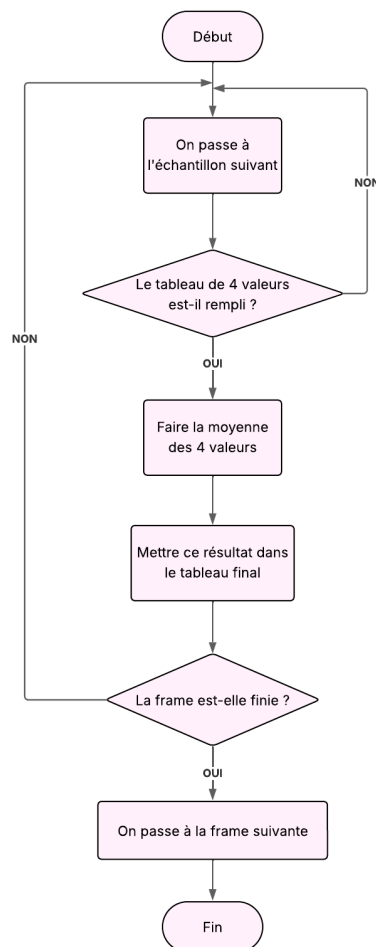


FIGURE 10 – Algorithme du subsampling de la fréquence

6.1.5 MFCC et DCT

La dernière étape du conditionnement du signal avant le réseau de neurones est l'application de l'algorithme MFC.

L'objectif premier de l'algorithme est de réduire la taille de la data caractérisant un signal sonore afin de par la suite pouvoir le reconnaître suffisamment rapidement.



FIGURE 11 – Etapes de l'algorithme MFCC (avec DCT)

Fenêtrage (Hamming Windows) :

Une fonction de fenêtre (par exemple, Hamming ou Hanning) est appliquée à chaque trame pour minimiser les discontinuités aux bords de la trame. La fenêtre est définie comme suit :

$$w[n] = 0,5(1 - \cos(2\pi \times \frac{n}{N-1}))$$

où $w[n]$ est la valeur de la fenêtre à l'échantillon n et N est la taille de la trame. Cette fonction est implémentée dans les méthodes `create_hamming_window()` ou encore `create_hanning_window()`.

Transformée de Fourier à court terme (STFT, Short Time Fourier Transform) :

La transformée de Fourier à court terme est appliquée à chaque trame pour obtenir le spectre de puissance. Cette étape permet de passer du domaine temporel au domaine fréquentiel.

Cette fonction est implémentée dans la méthode `apply_fft()`.

Filtres Mel :

Le spectre de puissance est filtré à l'aide d'un banc de filtres en triangle espacés selon l'échelle de fréquence Mel. L'échelle Mel est une échelle de fréquence perceptuelle qui tient compte de la manière dont l'oreille humaine perçoit les fréquences. La relation entre la fréquence Mel m et la fréquence linéaire f est la suivante :

$$m = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right) \quad (1)$$

Cette fonction est implémentée dans la méthode `create_mel_filter_bank()`.

Le logarithme de l'énergie de chaque filtre Mel est calculé. Cette étape permet de compresser les données en réduisant l'échelle dynamique.

Transformée en cosinus discrète (DCT) :

Enfin, la transformée en cosinus discrète est appliquée au spectre log-Mel pour obtenir les coefficients MFCC. Seuls les premiers k coefficients sont généralement conservés.

Cette fonction est implémentée dans la méthode `apply_dct()`.

Toutes ces fonctions étaient déjà codées sur le [GitHub du pôle électronique](#), ce qui a beaucoup aidé.

Ces tableaux de MFCC après passage de la DCT constituent notre entrée dans le réseau de neurones, que nous allons détailler ci-dessous.

6.2 Traitement des données

6.2.1 Introduction aux réseaux neuronaux

Pour commencer cette partie concernant le traitement des données, il est nécessaire de comprendre le fonctionnement d'un réseau de neurones convolutif.

Pour bien comprendre comment créer un réseau de neurones convolutif, il faut d'abord analyser sa structure. Un CNN est composé de deux parties, une partie axée sur la convolution et l'autre sur le réseau de neurones. La première partie est utile pour réduire le nombre d'entrées du réseau de neurones et accentuer les informations des matrices d'entrée, tandis que la seconde partie utilise les matrices transformées pour déterminer l'information contenue dans les matrices, à l'aide d'un réseau de neurones. Pour commencer, nous allons nous servir d'un exemple d'un jeu de valeur, utilisant le mot « bleu » et le mot « rouge », indépendant du projet. D'après les données disponibles sur le répertoire Github associé au cas d'exercice, une matrice d'un enregistrement de mot possède une taille de 48×13 , soit 624 entrées. Voici le graphique expliquant la partie convolutive de cet exemple :

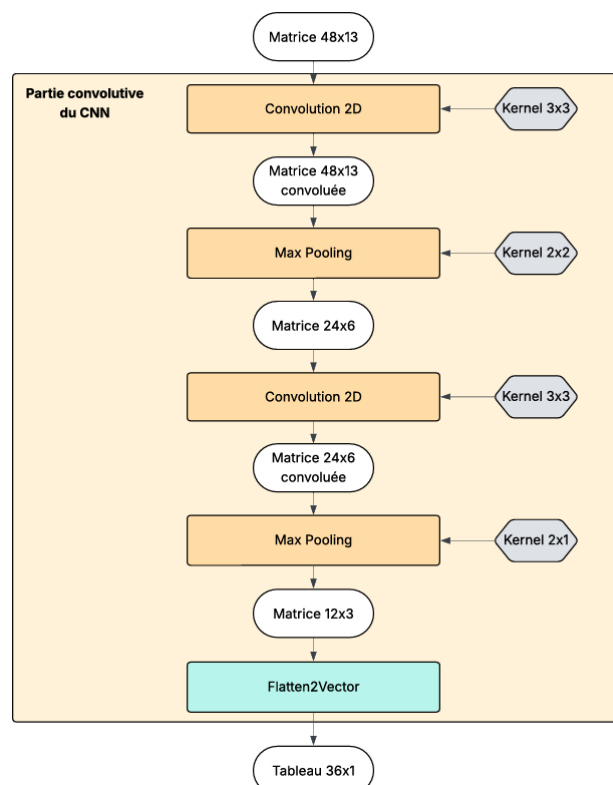


FIGURE 12 – Schéma explicatif de la partie convolutive du CNN appliqué à l'exemple « Bleu-Rouge »

La matrice d'origine est d'abord convoluée avec un noyau 3×3 décrit de cette façon :

0	0,1	0
0,1	0,8	0,1
0	0,1	0

TABLE 1 – Kernel de convolution 3x3 utilisé dans toutes les convolutions

Cette convolution permet de faire ressortir le contraste entre les données importantes de la matrice et leurs voisines. Un max-pooling est ensuite utilisé pour réduire la taille totale de la matrice, avec un kernel 2x2, qui divise par deux le nombre de lignes et le nombre de colonne, car cela prend quatre valeurs de la matrice, en format 2x2, et ne garde que la plus grande des quatre. Ces deux opérations sont ainsi répétées une seconde fois. Enfin, une fonction appelée *flatten2Vector()* transforme les matrices en tableau à une dimension contenant toutes les valeurs les unes après les autres. Ainsi, de nos 624 entrées, nous n'avons plus que 36 données par frame en sortie de notre partie convolutive.

Le réseau de neurones, suite logique de la partie de convolution, possède 36 entrées. Ce nombre de données par enregistrement est beaucoup plus intéressant que celui initial, car cela réduit la complexité du réseau neuronal. Voici un schéma du réseau neuronal :

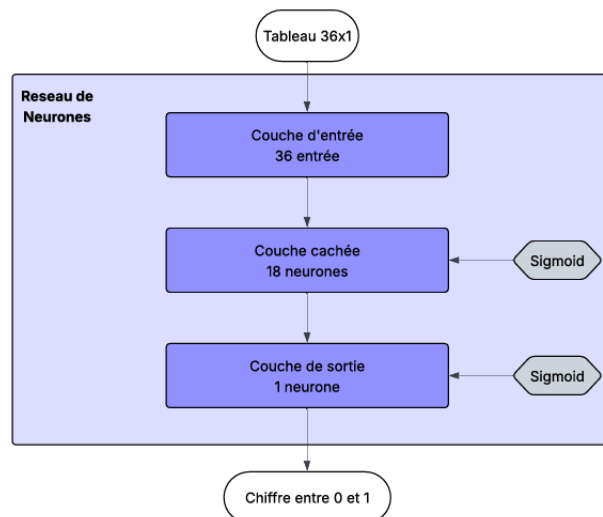


FIGURE 13 – Schéma explicatif de la partie du réseau neuronal appliqué à l'exemple « Bleu-Rouge »

On utilise ici la fonction d'activation Sigmoid, qui s'écrit sous la forme :

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Cette fonction permet de retourner une valeur entre 0 et 1, ce qui est très utile pour une partition binaire. On définit une couche cachée de 18 neurones, puis une couche de sortie composée d'un unique neurone, qui tend vers 0 si l'enregistrement est « bleu » ou 1 si l'enregistrement est « rouge ».

Pour entraîner notre modèle, nous avons utilisé la bibliothèque NeuralNetwork sur Arduino. En configurant notre script Arduino avec nos paramètres et en réalisant la partie convolutive avec le code prévu à cet effet, le réseau neuronal convolutif a été entraîné puis testé. Ainsi, la phase d'entraînement est symbolisée par la rétro-propagation pour le calcul des poids, où le réseau connaît la sortie attendue et compare son résultat avec cette dernière pour calculer l'erreur, tandis que la phase de test s'articule sur la prédiction de la sortie d'un enregistrement qu'il ne connaît pas. Après avoir effectué notre entraînement pendant 25 minutes et 2523 itérations pour obtenir un résultat pertinent, nous avons conclu

que nous aurions besoin d'entraîner notre modèle sur ordinateur avant de l'exporter sur Arduino, afin de réduire le temps d'entraînement.

Cette expérience nous a permis de comprendre comment établir la configuration d'un CNN, et comment l'utiliser sur Arduino à l'aide de la bibliothèque NeuralNetwork. Nous pouvons maintenant comprendre comment récupérer les enregistrements produits par Arduino pour permettre un entraînement personnalisé.

6.2.2 Echanges des données inter-plateformes

Dans un souci d'optimisation et de rapidité, nous avons fait le choix d'entraîner notre réseau neuronal sur Python à l'aide de la bibliothèque TensorFlow, puis d'en importer les poids calculés dans la carte Arduino. De ce fait, nous avons dû mettre en place un système de normalisation permettant d'échanger les données traitées par la carte Arduino (formatées pour le langage C++) et celles traitées par le script Python permettant d'entraîner le modèle.

Comme expliqué précédemment, le réseau neuronal configuré sur TensorFlow prend en données d'entraînement un tableau contenant 200 mots, chacun de 132 frames elles-mêmes composées de 6 coefficients MFC après passage de l'algorithme de la DCT. Pour transmettre ces informations à l'ordinateur, nous avons mis en place une série d'instructions permettant d'afficher chaque mot enregistré par le système suivant le format des tableaux de la bibliothèque numpy.

```
13:26:56.234 -> [[41.17, 4.77, 3.01, 3.01, 1.92, 1.33],
13:26:56.234 -> [24.46, 3.53, 1.29, 2.70, 1.47, 0.78],
13:26:56.234 -> [22.32, 4.36, 2.47, 2.14, 1.42, 0.59],
13:26:56.234 -> [22.57, 4.63, 2.42, 3.35, 1.56, 0.95],
13:26:56.234 -> [24.47, 3.45, 1.03, 2.78, 1.27, 0.56],
13:26:56.234 -> [23.74, 4.35, 1.45, 2.39, 1.66, 1.29],
13:26:56.234 -> [24.00, 2.70, 1.25, 3.00, 1.67, 0.92],
13:26:56.234 -> [24.06, 4.19, 1.84, 1.97, 1.69, 0.74],
13:26:56.234 -> [23.01, 3.90, 2.03, 3.32, 2.45, 1.14],
13:26:56.234 -> [23.99, 3.45, 1.71, 1.84, 1.65, 0.65],
13:26:56.234 -> [22.78, 3.07, 2.09, 2.67, 1.78, 0.82],
13:26:56.234 -> [23.03, 4.09, 1.94, 2.55, 1.39, 1.26],
13:26:56.234 -> [22.64, 3.72, 1.87, 2.98, 1.78, 0.98],
13:26:56.234 -> [24.72, 4.23, 1.42, 2.76, 2.19, 0.95],
13:26:56.234 -> [24.59, 4.31, 2.40, 3.60, 2.21, 1.30],
13:26:56.234 -> [25.04, 4.16, 1.76, 3.58, 1.36, 1.13],
13:26:56.234 -> [25.31, 4.02, 1.41, 3.59, 1.86, 1.42],
13:26:56.234 -> [27.51, 5.83, 3.16, 2.74, 1.45, 0.78],
13:26:56.234 -> [29.63, 5.97, 1.72, 2.73, 1.94, 1.07],
13:26:56.234 -> [25.94, 5.01, 2.43, 2.60, 1.82, 0.92],
13:26:56.234 -> [25.19, 5.39, 2.41, 2.89, 2.76, 1.34],
13:26:56.234 -> [25.21, 4.46, 2.53, 2.90, 1.82, 0.95],
13:26:56.234 -> [29.08, 6.38, 1.41, 2.93, 1.83, 0.75],
13:26:56.234 -> [30.09, 6.11, 1.98, 2.77, 1.53, 0.70],
13:26:56.234 -> [29.40, 5.67, 2.09, 2.48, 1.43, 0.72],
13:26:56.234 -> [26.66, 5.39, 2.85, 2.79, 1.43, 0.80],
13:26:56.234 -> [26.43, 5.54, 2.13, 2.45, 1.50, 1.07],
13:26:56.234 -> [27.92, 5.36, 2.71, 3.12, 1.81, 1.13],
```

FIGURE 14 – Données normalisées sous forme de tableau numpy en sortie de l'Arduino DUE

Ces valeurs sont alors récupérées par le logiciel PuTTY qui les écrit dans des fichiers .txt prévus à cet effet. Enfin, ces fichiers sont transmis au script Python du modèle.

Concernant les poids calculés par le modèle TensorFlow, ceux-ci sont directement écrits dans la mémoire de la carte Arduino DUE à son chargement. L'avantage de procéder de la sorte est notable : toute la phase d'entraînement du réseau neuronal est déléguée à un PC, considérablement plus puissant que la carte Arduino, ce qui nous permet un gain de temps important ainsi qu'une légèreté notable dans le code du microcontrôleur. En effet, l'entraînement sur ce support aurait impliqué une charge considérable

sur la mémoire de la carte compte tenu du nombre de données d'entraînement et de tests, ainsi qu'un temps de convergence immense.

6.2.3 Classification des enregistrements

Cette partie sera consacrée à la classification des enregistrements entre deux mots : « Thais » et « Guy-Cha ». En effet, l'objectif final de notre projet étant de réaliser un jeu « qui de nous 4 », cette étape est donc une étape intermédiaire nommée « qui de nous 2 ».

Fort de notre expérience avec l'exemple « Bleu-Rouge », nous avons d'abord voulu créer la structure de notre réseau neuronal. Sachant qu'un enregistrement est composé de 132 frames, qui sont elles-mêmes composées de 6 coefficients DCT, il a fallu revoir la convolution appliquée afin de réduire le nombre d'entrées du réseau neuronal. Voici un schéma récapitulatif :

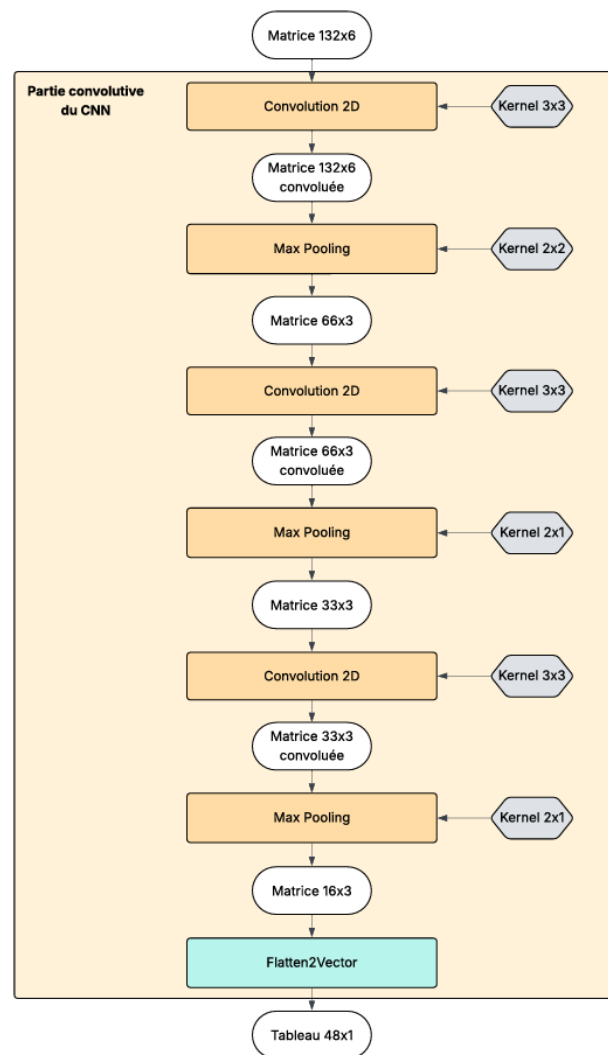


FIGURE 15 – Schéma explicatif de la partie de convolution du CNN

La matrice d'entrée de 132 par 6 est convoluée par le même kernel que celui de l'exemple « Bleu-Rouge », à savoir :

0	0,1	0
0,1	0,8	0,1
0	0,1	0

TABLE 2 – Kernel de convolution 3x3 utilisé dans toutes les convolutions

Il permet d'augmenter le contraste entre la valeur de la donnée et ses voisines. Cette matrice convoluée est ensuite passée dans un filtre max pooling avec un kernel de dimension 2x2, ce qui réduit la longueur et la largeur du tableau par deux. On effectue ensuite la même opération de convolution, mais le second max-pooling prend cette fois un kernel 2x1, pour ne pas diminuer le nombre de coefficients DCT par frame à 1, et ainsi garder 3 coefficients par frame. On réitère une convolution et un max-pooling avec un kernel 2x1 pour obtenir une matrice 16x3, soit 48 valeurs.

D'un point de vue technique, il a fallu implémenter de nouvelles convolutions et maxpooling dans la bibliothèque « cnn » fournie dans le sujet avec les particularités énoncées. De plus, nous avons choisi de faire l'entraînement sur Tensorflow. Ces fonctions convolutives ont donc été recodées sur Tensorflow pour obtenir la même convolution, plutôt que celle intégrée dans cette bibliothèque Python. En effet, Python et C++ (utilisés par Arduino) ne possèdent pas le même nombre de chiffres significatif après la virgule sur des nombres de type float, ce qui change les calculs réalisés sur des données précises, comme des coefficients DCT.

On peut alors s'intéresser au réseau neuronal utilisé. Voici son schéma explicatif :

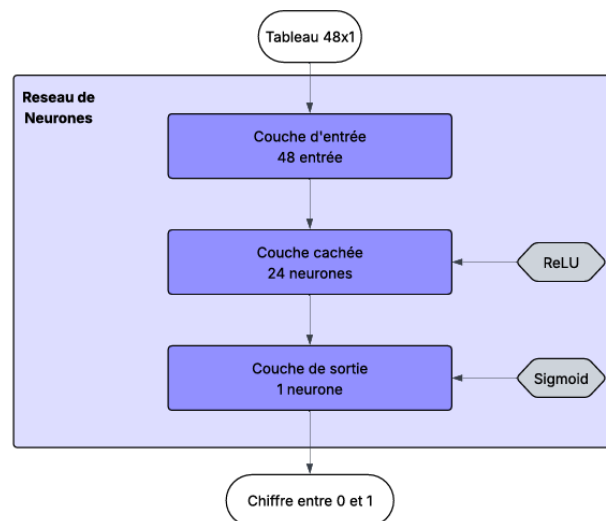


FIGURE 16 – Schéma explicatif de la partie du réseau neuronal à deux sorties

Le réseau neuronal possède donc 48 entrées, une couche cachée de 24 neurones utilisant la fonction d'activation ReLU et une couche de sortie d'un neurone utilisant la fonction d'activation Sigmoid. La plus grande différence avec le réseau neuronal de l'exemple réside dans la fonction d'activation des neurones de la couche cachée. La fonction d'activation Rectified Linear Unit (ou ReLU) est très intéressante pour un réseau de neurones convolutif. Elle ne nécessite pas de calculs exponentiels et supprime le problème du gradient, qui peut s'annuler quand le réseau de neurones devient complexe. voici sa formule :

$$f(x) = \begin{cases} x, & \text{si } x \geq 0 \\ 0, & \text{si } x < 0 \end{cases}$$

Cette nouvelle fonction d'activation est déjà implémenté dans les bliblioteques neuralNetwork sur Arduino et Tensorflow sur Python, il est donc aisé de l'utiliser dans notre reseau de neurone.

La structure de notre CNN est donc identique sur Tensorflow et sur Python. Nous pouvons alors commencer l'entraînement de notre réseau de neurones. Pour les paramètres de l'entraînement, nous avons fait le choix d'utiliser le compilateur Adam intégré à la bibliothèque, et de modifier son pas d'apprentissage à 0,005.

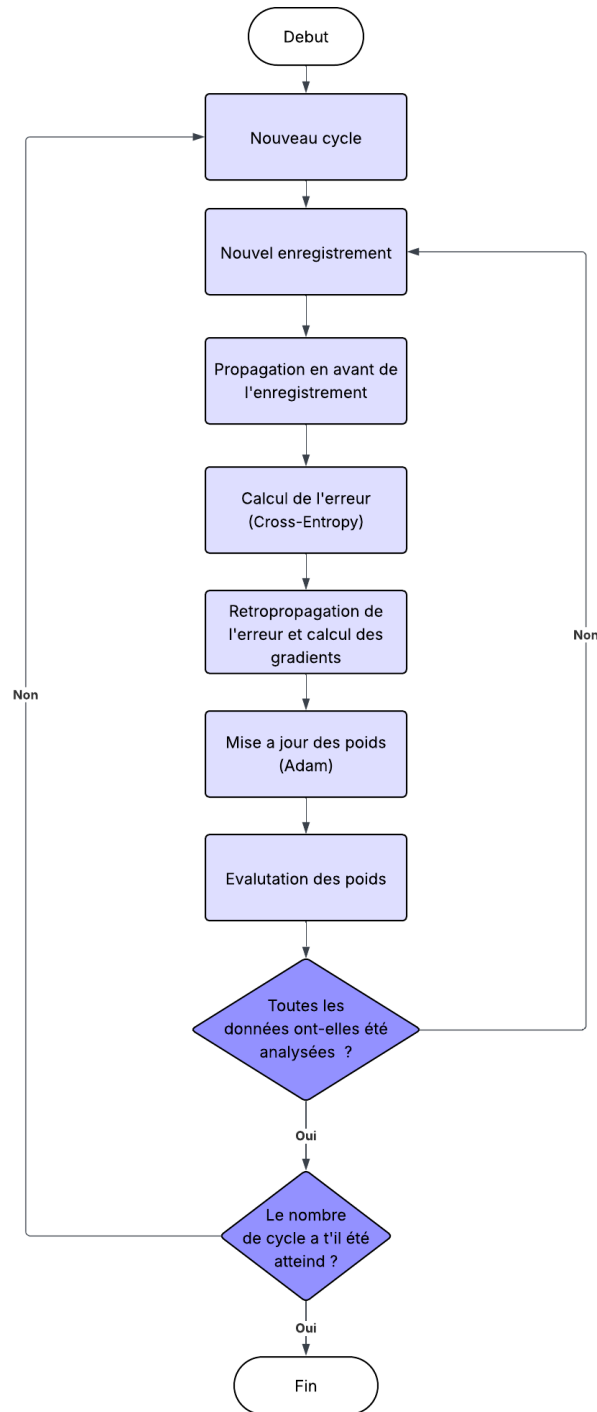


FIGURE 17 – Algorithme d’entraînement du NN sur Tensorflow

Cet algorithme explique le fonctionnement de l’entraînement du réseau neuronal sur Tensorflow. Un cycle, ou epoch en anglais, correspond à l’analyse de l’ensemble des données de test. Pour chaque enregistrement, l’algorithme propage en avant les coefficients DCT convolués, puis calcule l’erreur en utilisant la fonction *Cross-Entropy()*. Cette fonction de perte, à l’instar de la MSE, utilise la valeur calculée et la valeur réelle comme paramètre. Voici sa formule :

$$L = - \sum_i y_i \log(\hat{y}_i)$$

où y_i est la valeur réelle de la classe i et $\hat{y}_i$ est la probabilité prédite pour cette classe. Les erreurs sont ensuite rétropropagées dans le réseau neuronal pour calculer les gradients. Enfin, les poids sont mis à jour par le compilateur Adam (Adaptive Moment Estimation) qui adapte dynamiquement le taux d'apprentissage en fonction de l'historique des gradients tel que :

$$w_{t+1} = w_t - \frac{\alpha m_t}{\sqrt{v_t} + \epsilon}$$

où w_t est le poids à l'instant t , α est le taux d'apprentissage, m_t est le moment de premier ordre (moyenne des gradients), v_t est le moment de second ordre (variance des gradients) et ϵ est un petit facteur pour éviter la division par zéro. Le nouveau jeu de poids est ensuite testé avec une partie des données d'entraînement, puis l'apprentissage passe à la prochaine donnée. Une fois toutes les données testées, l'entraînement passe au cycle suivant.

Après cela, nous avons exporté les poids et biais obtenus pour chaque neurone sur Arduino, afin de reproduire le réseau neuronal construit. Pour valider sa bonne importation, nous avons reproduit les mêmes tests sur Arduino et sur Python et observé les résultats, qui étaient alors identiques. Le temps d'entraînement est de 543,650 secondes, soit 9 minutes. Cette durée est 2,78 fois plus petite que lors de l'entraînement du cas d'entraînement sur Arduino.

TensorFlow est donc un outil puissant capable de raccourcir la durée d'entraînement tout en obtenant des résultats identiques. Ce réseau neuronal est prometteur et valide complètement le cahier des charges. L'objectif va maintenant être de diversifier notre nombre de sorties, en passant de deux à quatre.

6.2.4 Élargissement de la classification

Pour élargir la classification à quatre mots distincts, il faut alors se questionner sur le fonctionnement de la sortie d'un réseau neuronal. Dans la partie précédente, nous utilisons un neurone en sortie avec une fonction d'activation sigmoïde, qui nous permettait de séparer deux cas. Bien que fonctionnelle, cette méthode est donc inutilisable pour plus de deux cas. Notre solution a été d'utiliser la fonction Softmax. La fonction SoftMax est utile dans un réseau de neurones pour classer des données en plusieurs catégories. Elle est définie telle que :

$$S_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Avec S_i le résultat en sortie de la fonction, x_i la valeur en entrée de ce neurone et $\sum_j e^{x_j}$ la somme des valeurs en entrées de chaque neurone de la couche. Cette fonction d'activation permet donc d'avoir une probabilité en sortie, et facilite donc la classification des données. Voici alors le nouveau réseau neuronal :

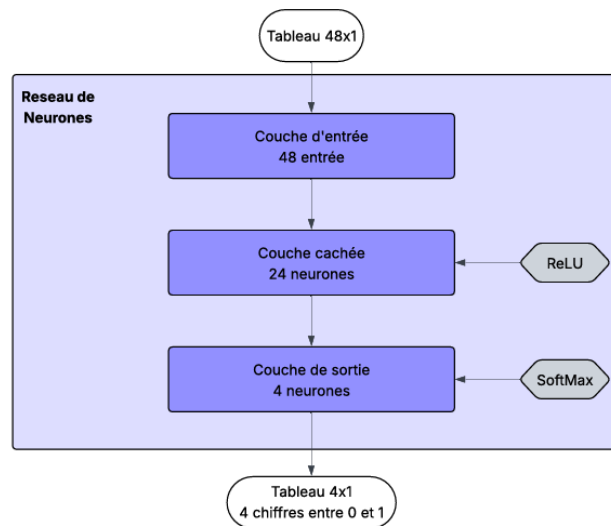


FIGURE 18 – Schéma explicatif de la partie du réseau neuronal à quatre sorties

Ce nouveau réseau neuronal possède donc quatre sorties, formant une partition de l'univers. Le reste du réseau de neurones est le même que le cas avec deux sorties possibles. Nous avons fait le choix de changer le mot « Thomas » pour « Toto » pour deux raisons : d'abord pour souligner l'aspect familial du rendu final du projet, mais aussi pour simplifier la classification entre Thais, Guy-cha et Thomas.

Pour intégrer notre classification à notre code général, il suffit alors de récupérer l'enregistrement produit, de l'envoyer à travers le réseau neuronal pour obtenir une prédiction, puis d'allumer la LED correspondante en fonction de la réponse obtenue.

7 Tests et validation

7.1 Acquisition du signal

7.1.1 Numériser le signal audio

Pour valider que la numérisation du signal est opérationnelle, nous nous sommes aidés d'un oscilloscope fourni par l'ECE. Cet appareil de mesure permet de visualiser et d'analyser des signaux électriques en fonction du temps.

Dans notre projet, nous avons utilisé l'oscilloscope comme outil pour nous assurer que nous échantillonnions à la bonne fréquence.

Voici comment nous avons procédé :

Tout d'abord, nous avons relié un générateur de tension à notre carte Arduino Due, sur l'entrée analogique correspondant à l'ADC (A0), dans le but de générer en entrée un signal électrique d'amplitude 1V et de fréquence 1 kHz.

Ce signal est ensuite traité par le microcontrôleur de la Arduino pour être échantillonné à 32 kHz.

Afin de voir si ce dispositif fonctionne correctement, nous configurons le DAC afin que celui-ci sorte les valeurs analogiques à partir des valeurs numériques fournies par l'ADC. On analyse alors cette sortie à l'aide de l'oscilloscope.

Nous obtenons alors la figure suivante :



FIGURE 19 – Comparaison des fréquences des signaux d'entrée et sortie à 1kHz

Nous observons 2 signaux électriques.

- Le violet représente le signal d'entrée généré par le générateur de tension d'amplitude 1V. C'est une valeur arbitraire mais réfléchiée dans l'intervalle de tension autorisé de l'Arduino Due à savoir entre 0 et 3,3V. Ce signal a une fréquence de 1kHz.
- Le signal jaune correspond au signal de sortie après échantillonnage.

Avant même d'effectuer des calculs, nous observons que le signal jaune est segmenté par de nombreux points indiquant un échantillonnage à une fréquence élevée.

Cependant cette analyse n'est pas fiable pour s'assurer que les caractéristiques du signal d'origine (comme la fréquence) sont préservées après échantillonnage. Nous avons donc procédé à un calcul qui

vérifie l'échelle temporelle grâce aux précieuses informations fournies par l'oscilloscope. Nous savons que la fréquence F se calcule à l'aide de la formule suivante :

$$F = \frac{1}{T}$$

La période T correspond à l'intervalle de temps mesuré entre deux mêmes motifs. Ici un carreau est égal 500 μ s. Or, nous comptons deux carreaux entre deux motifs donc $T = 500 \times 2 = 0.001$ s. Nous effectuons donc $F = \frac{1}{0.001}$. Ainsi la fréquence du signal jaune égale celle du signal violet, 1kHz.

Nous venons de valider que le traitement du signal numérique reflète fidèlement le signal analogique d'origine.

Maintenant nous devons vérifier que notre fréquence d'échantillonnage est exacte.

Afin de valider les calculs des différentes valeurs du registre de comparaison, on adapte le programme pour qu'un état HAUT soit envoyé sur la pin digitale 2 lors de l'interruption timer. La lecture de cette pin à l'oscilloscope nous permet alors de confirmer les calculs effectués précédemment et de mesurer la fréquence de l'interruption. Voici le résultat obtenu :

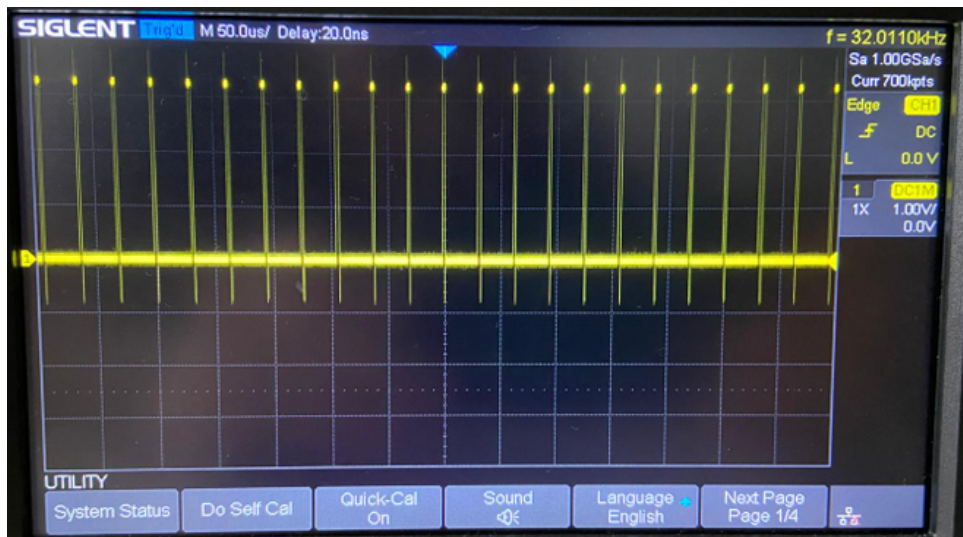


FIGURE 20 – Analyse sur oscilloscope de la fréquence d'échantillonnage à 32kHz

Sur la figure, l'oscilloscope nous apporte la confirmation que nous effectuons un échantillonnage à 32kHz comme nous pouvons le lire en haut à droite.

Les deux validations que nous venons de faire permettent de vérifier que la condition de Nyquist est bien respectée. Nous utilisons ce théorème pour éviter le repliement de spectre et donc la perte d'information. La fréquence d'échantillonnage f_e doit être au moins deux fois supérieure à la fréquence maximale f_{max} du signal.

$$f_e \geq 2f_{max}$$

Dans notre cas, avec un signal de fréquence 1 kHz, une fréquence d'échantillonnage de 32 kHz est largement suffisante pour éviter le repliement.

7.1.2 Conditionnement du signal

Afin de valider le conditionnement du signal, nous nous sommes aidés du logiciel de traitement sonore Audacity. Cette outil nous permet d'importer nos propres enregistrements sous forme de données brutes. Cela signifie que notre fichier audio n'as pas besoin d'avoir un format audio standard comme .MP3 ou .WAV.

Cette particularité nous est très utile car nous sauvegardons nos données enregistrées par le micro grâce au logiciel PuTTY dans le format .bin. Il s'agit d'un fichier binaire, ce qui signifie qu'il stocke les données sous forme d'octets.

Lorsque l'importation du fichier a été effectuée, nous avons observé que la piste affichée sur Audacity était correctement audible à 32 kHz.

Pour confirmer le bon fonctionnement du filtre, nous avons du nous intéresser à son spectre audio. Le spectre est une représentation graphique des fréquences présentes dans une portion d'un enregistrement audio. L'axe horizontal représente les fréquences, allant des basses fréquences à gauche aux hautes fréquences à droite. Quant à l'axe vertical, il représente l'amplitude ou l'intensité de chaque fréquence exprimée en décibels (dB).

Voici le spectre du signal que nous obtenons :

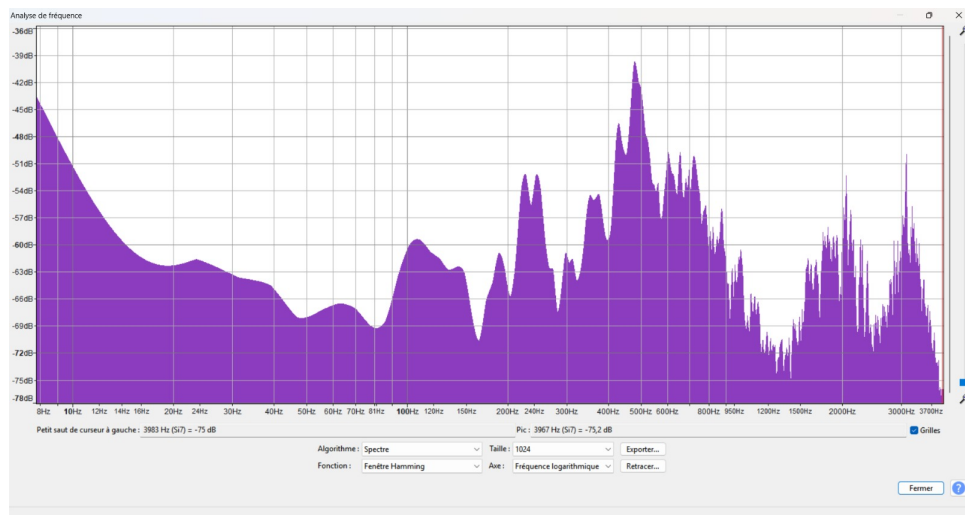


FIGURE 21 – Graphique du spectre du signal filtré

Le cahier des charges requiert de notre filtre passe-bas qu'il atténue d'au moins 30dB toutes les fréquences supérieures à 4kHz. Or nous observons sur l'axe des abscisses que les fréquences vont d'environ 8Hz à 3980kHz.

Pour calculer l'atténuation en décibels exact, nous devons identifier les valeurs de la puissance émise P_e et de la puissance reçue P_s d'après le graphique afin d'utiliser la formule suivante :

$$dB = 10 \log\left(\frac{P_e}{P_s}\right)$$

La puissance émise correspond à la valeur de la puissance à une fréquence donnée où le signal est le plus fort. Ici, nous remarquons que la fréquence où le signal le plus fort est 500Hz. Et sur l'axe des ordonnées, nous apparaît la valeur -39 dB.

Nous procédons à la même analyse pour la puissance reçue, correspondant à la puissance à une fréquence donnée où le signal est le plus faible (ici 3983Hz). Cette fois-ci nous apparaît la valeur -75 dB.

Effectuons la conversion des dB en puissance :

$$P_e = 10^{\frac{-39}{10}} = 1.2589 \times 10^{-4}$$

$$P_s = 10^{\frac{-75}{10}} = 3.1623 \times 10^{-8}$$

Calculons alors l'atténuation :

$$dB = 10 \log \left(\frac{1.2589 \times 10^{-4}}{3.1623 \times 10^{-8}} \right)$$

$$dB = 10 \log(3981.07)$$

$$dB = 10 \times 3.6$$

$$dB = 36 \text{ dB}$$

Nous retrouvons bien une atténuation de -36dB.

Le spectre obtenu est donc effectivement un filtre passe-bas qui atténue d'au moins 30dB les fréquences supérieures à 4kHz.

A titre de comparaison, voici le spectre audio du même signal non-filtré :

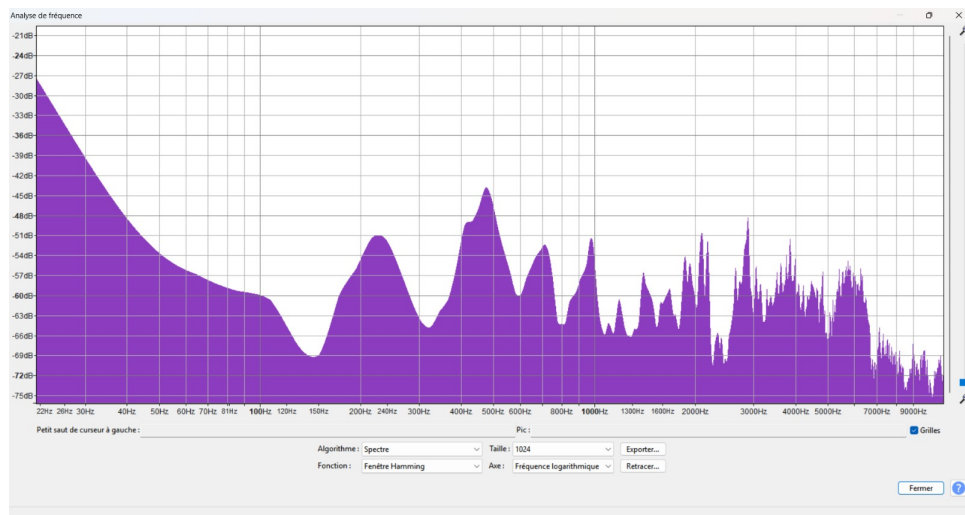


FIGURE 22 – Graphique du spectre du signal non filtré

Si nous faisons les mêmes calculs d'atténuations nous obtenons -23dB et nous observons que les fréquences vont de 20Hz à plus de 9000kHz.

Concernant les performances du filtre, nous avons mesuré le temps pris par le filtre pour s'appliquer aux données d'entrée. Après optimisations (détaillées précédemment), nous avons pu atteindre les exigences imposées en terme de performances. Nous validons donc un temps de filtrage inférieur à 31 μ s :

```
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 28
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 30
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 28
Temps de conversion (en us) : 29
Temps de conversion (en us) : 29
Temps de conversion (en us) : 28
Temps de conversion (en us) : 30
Temps de conversion (en us) : 29
```

FIGURE 23 – Validation du temps de filtrage inférieur à 31 μ s

Enfin, on valide également le fonctionnement du buffer circulaire en affichant sa sortie sur 10 cycles :

```
1603 27008 1621 0 29559 14381 29559 14381 -16046 -16432 16673 8 0 -32768 2321 -17323 -16098 -27705 2545 16257 9157 8 23754 16448 -1 32751 8699 16361 0 0 -1 -1 271 0 -1 -1 0 0 21652 8199 0 0
1603 1645 1621 0 29559 14381 29559 14381 -16046 -16432 16673 8 0 -32768 2321 -17323 -16098 -27705 2545 16257 9157 8 23754 16448 -1 32751 8699 16361 0 0 -1 -1 271 0 -1 -1 0 0 21652 8199 0 0
1603 1645 1683 0 29559 14381 29559 14381 -16046 -16432 16673 8 0 -32768 2321 -17323 -16098 -27705 2545 16257 9157 8 23754 16448 -1 32751 8699 16361 0 0 -1 -1 271 0 -1 -1 0 0 21652 8199 0 0
1603 1645 1683 1280 29559 14381 29559 14381 -16046 -16432 16673 8 0 -32768 2321 -17323 -16098 -27705 2545 16257 9157 8 23754 16448 -1 32751 8699 16361 0 0 -1 -1 271 0 -1 -1 0 0 21652 8199 0 0
1603 1645 1683 1280 1683 14381 29559 14381 -16046 -16432 16673 8 0 -32768 2321 -17323 -16098 -27705 2545 16257 9157 8 23754 16448 -1 32751 8699 16361 0 0 -1 -1 271 0 -1 -1 0 0 21652 8199 0 0
1603 1645 1683 1280 1683 2239 29559 14381 -16046 -16432 16673 8 0 -32768 2321 -17323 -16098 -27705 2545 16257 9157 8 23754 16448 -1 32751 8699 16361 0 0 -1 -1 271 0 -1 -1 0 0 21652 8199 0 0
1603 1645 1683 1280 1683 2239 1108 14381 -16046 -16432 16673 8 0 -32768 2321 -17323 -16098 -27705 2545 16257 9157 8 23754 16448 -1 32751 8699 16361 0 0 -1 -1 271 0 -1 -1 0 0 21652 8199 0 0
1603 1645 1683 1280 1683 2239 1108 1375 -16046 -16432 16673 8 0 -32768 2321 -17323 -16098 -27705 2545 16257 9157 8 23754 16448 -1 32751 8699 16361 0 0 -1 -1 271 0 -1 -1 0 0 21652 8199 0 0
1603 1645 1683 1280 1683 2239 1108 1375 2043 -16432 16673 8 0 -32768 2321 -17323 -16098 -27705 2545 16257 9157 8 23754 16448 -1 32751 8699 16361 0 0 -1 -1 271 0 -1 -1 0 0 21652 8199 0 0 -7
1603 1645 1683 1280 1683 2239 1108 1375 2043 1753 16673 8 0 -32768 2321 -17323 -16098 -27705 2545 16257 9157 8 23754 16448 -1 32751 8699 16361 0 0 -1 -1 271 0 -1 -1 0 0 21652 8199 0 0 -7
1603 1645 1683 1280 1683 2239 1108 1375 2043 1753 1222 8 0 -32768 2321 -17323 -16098 -27705 2545 16257 9157 8 23754 16448 -1 32751 8699 16361 0 0 -1 -1 271 0 -1 -1 0 0 21652 8199 0 0 -7
1603 1645 1683 1280 1683 2239 1108 1375 2043 1753 1222 2201 0 -32768 2321 -17323 -16098 -27705 2545 16257 9157 8 23754 16448 -1 32751 8699 16361 0 0 -1 -1 271 0 -1 -1 0 0 21652 8199 0 0 -7
1603 1645 1683 1280 1683 2239 1108 1375 2043 1753 1222 2201 2941 -32768 2321 -17323 -16098 -27705 2545 16257 9157 8 23754 16448 -1 32751 8699 16361 0 0 -1 -1 271 0 -1 -1 0 0 21652 8199 0 0
```

FIGURE 24 – Validation du bon fonctionnement du buffer circulaire sur 10 cycles

7.1.3 Subsampler le signal

Nous cherchons dans cette section à valider qu'un sous-échantillonnage a bien été effectué sur notre signal.

Nous rappelons que nous avons effectué un premier échantillonnage à 32kHz et comme nous l'avons expliqué dans la partie développement du rapport, un sous-échantillonnage (subsampling) à 8 kHz était nécessaire.

Compte tenu de notre précédente validation sur l'oscilloscope (ET1), nous savions que notre signal était correctement échantillonné à 32kHz. Dès lors nous avons importé ce signal sur Audacity et nous avons constaté que la piste audio zoomée à très grande échelle n'est autre qu'une succession de points.

Ainsi, pour visualiser le sous-échantillonnage à 8kHz, nous avons pensé à comparer la piste audio du signal échantillonné à 32kHz et celui à 8kHz. Voici ce que nous obtenons :

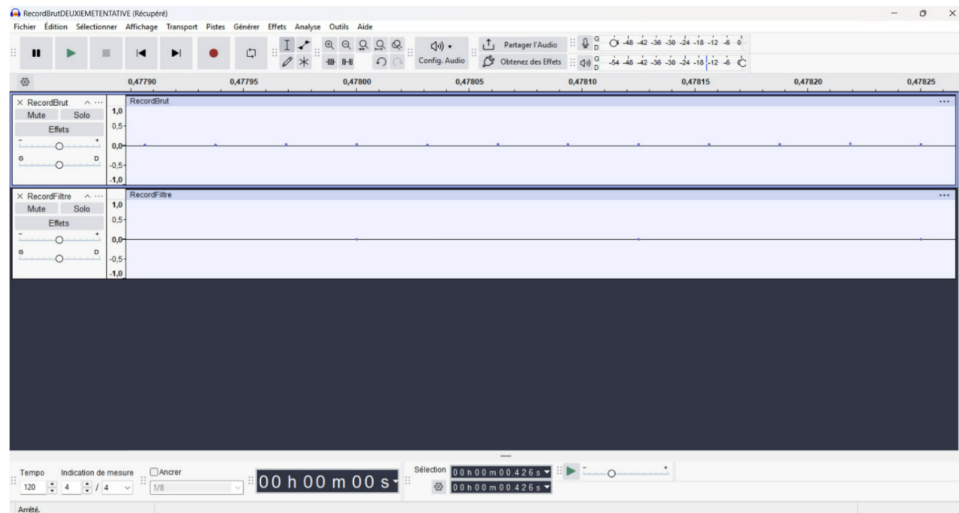


FIGURE 25 – Analyse du signal échantillonné à 8kHz

La première piste audio correspond à notre signal échantillonné à 32kHz et la seconde au sous-échantillonnage à 8kHz.

Nous savons qu'effectuer un sous-échantillonnage à 8kHz d'un signal de 32kHz revient à sélectionner un échantillon sur quatre.

Or sur la figure nous observons qu'un point (un échantillon) du signal audio sous-échantillonné à 8kHz correspond à quatre points du signal audio échantillonné à 32kHz. Mathématiquement, c'est exactement le résultat attendu :

$$\frac{32}{4} = 8$$

Ainsi, nous validons le sous-échantillonnage du signal à 8 kHz.

7.1.4 Écouter et valider l'enregistrement

Afin de valider l'enregistrement du microphone, nous avons mis en place une communication série entre la carte Arduino et un PC muni du logiciel PuTTY afin de récupérer les informations traitées par la carte. Ces données, stockées dans un fichier binaire, sont ensuite lues dans le logiciel Audacity. Nous validons alors la restitution d'un enregistrement du mot « Electronique » :

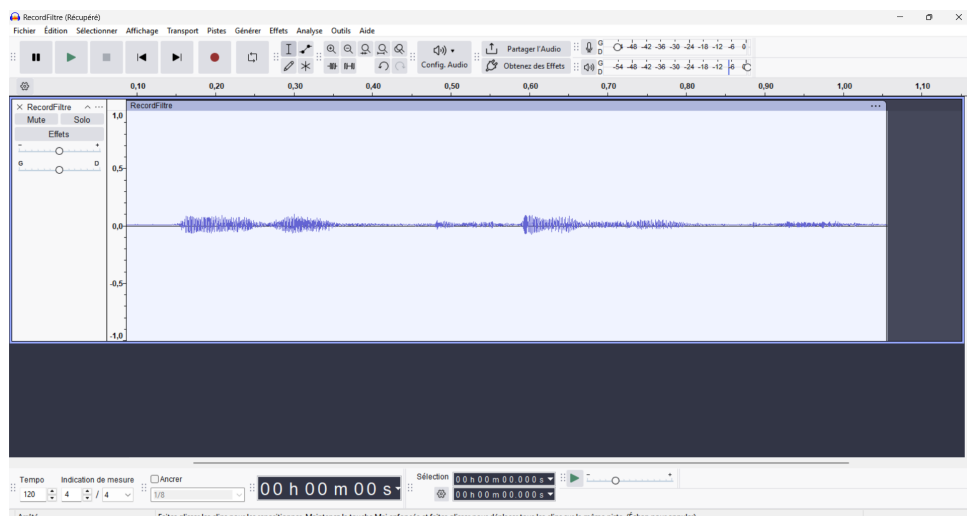
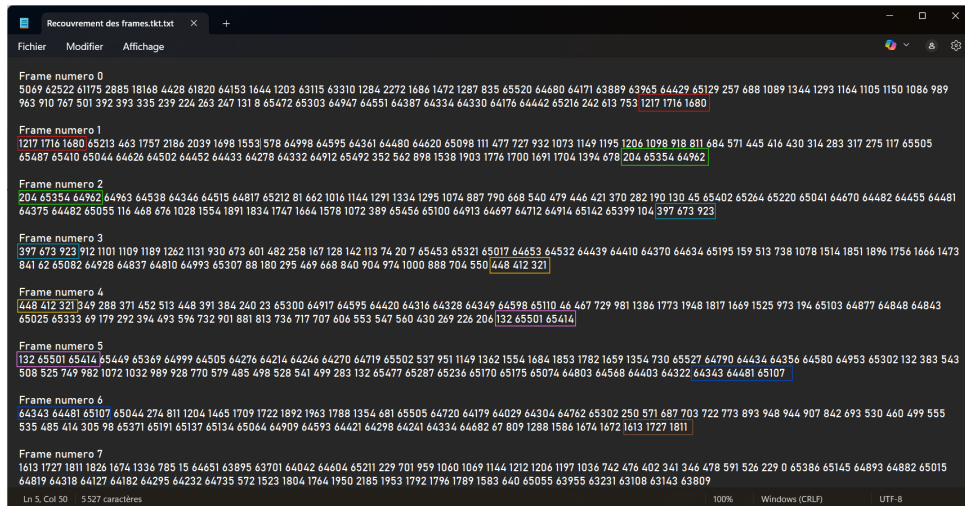


FIGURE 26 – Spectre du mot « Electronique »

Cet enregistrement est également écoutable à cette adresse : <https://audio.com/lucie-daix/audio/recordfiltre>

7.1.5 Caractériser le timbre vocal

Afin de valider l'algorithme de recouvrement, nous avons fait le choix d'analyser les données numériques plutôt que les spectres individuels des nombreuses frames qui composent un enregistrement. On note également que le recouvrement est effectué dans le même temps que le subsampling.



```
Recouvrement des frames.txt
Fichier  Modifier  Affichage

Frame numero 0
5069 62522 61175 2885 18168 4428 61820 64153 1644 1203 63115 63310 1284 2272 1686 1472 1287 835 65520 64680 64171 63889 63965 64429 65129 257 688 1089 1344 1293 1164 1105 1150 1086 989
963 910 767 501 392 393 335 239 224 263 247 131 8 65472 65303 64947 64551 64387 64334 64330 64176 64442 65216 242 613 753 1217 1716 1680

Frame numero 1
1217 1716 1680 65213 463 1757 2186 2029 1698 1553 578 64998 64595 64341 64480 64420 65098 111 477 727 932 1073 1149 1195 1206 1098 918 811 684 571 445 416 430 314 283 317 275 117 65505
65487 65410 65044 64626 64502 64452 64433 64278 64332 64912 65472 352 562 898 1538 1903 1776 1700 1691 1704 1394 678 204 65354 64962

Frame numero 2
204 65354 64962 64963 64538 64346 64515 64817 65212 81 662 1016 1144 1291 1334 1295 1074 887 790 668 540 479 446 421 370 282 190 130 45 65402 65264 65220 65041 64670 64482 64455 64481
64375 64482 65055 116 468 676 1028 1554 1891 1834 1747 1664 1578 1072 389 65456 65100 64913 64697 64712 64914 65142 65399 104 397 673 923

Frame numero 3
397 673 923 912 1101 1109 1189 1262 1131 930 673 401 482 258 167 128 142 113 74 20 7 65453 65321 65017 64653 64532 64439 64410 64370 64634 65195 159 513 738 1078 1514 1851 1896 1756 1666 1473
841 62 65082 64928 64837 64810 64993 65307 88 180 295 469 668 840 904 974 1000 888 704 550 448 412 321

Frame numero 4
448 412 321 349 288 371 452 513 448 391 384 240 23 65300 64917 64595 64420 64316 64328 64349 64598 65110 46 467 729 981 1386 1773 1948 1817 1669 1525 973 194 65103 64877 64848 64843
65025 65333 67 179 292 394 493 596 732 901 881 813 736 717 707 606 553 547 560 430 269 226 206 132 65501 65414

Frame numero 5
132 65501 65414 65449 65369 64999 64505 64276 64214 64246 64270 64719 65502 537 951 1149 1362 1554 1684 1853 1782 1659 1354 730 65527 64790 64434 64356 64580 64953 65302 132 383 543
508 525 749 982 1072 1032 989 928 770 579 485 498 528 541 499 283 132 65477 65287 65236 65170 65175 65074 64803 64568 64403 64322 64343 64481 65107

Frame numero 6
64343 64481 65107 65044 274 811 1204 1465 1709 1722 1892 1963 1788 1354 681 65505 64720 64179 64029 64304 64762 65302 250 571 487 703 722 773 893 948 944 907 842 693 530 460 499 555
535 485 414 305 98 65371 65191 65137 65134 65064 64909 64593 64421 64298 64241 64334 64482 67 809 1288 1586 1674 1672 1613 1727 1811

Frame numero 7
1613 1727 1811 1826 1674 1336 785 15 64651 63895 63701 64042 64604 65211 229 701 959 1060 1069 1144 1212 1206 1197 1036 742 476 402 341 346 478 591 526 229 0 65386 65145 64893 64882 65015
64819 64318 64127 64182 64295 64232 64735 572 1523 1804 1764 1950 2185 1953 1792 1796 1789 1583 640 65055 63955 63231 63108 63143 63809

Ln 5, Col 50  5327 caractères
100%  Windows (CRLF)  UTF-8
```

FIGURE 27 – Validation du recouvrement des frames

Comme expliqué plus haut, le recouvrement est initialement de 12 échantillons avant d'être abaissé à 3 lors du subsampling. On remarque bien la duplication des dernières valeurs de chaque frame au début de la suivante.

Concernant le fonctionnement de l'algorithme de calcul des MFCC, on en analyse successivement chaque étape :

Pré-emphase En modifiant légèrement la bibliothèque `arduinoMFCC`, on implémente le même algorithme que pour la validation de l'enregistrement audio (via PuTTY et Audacity) après l'application du pré-emphase. Cela nous permet alors d'analyser le spectre de l'enregistrement avant et après le pré-emphase :

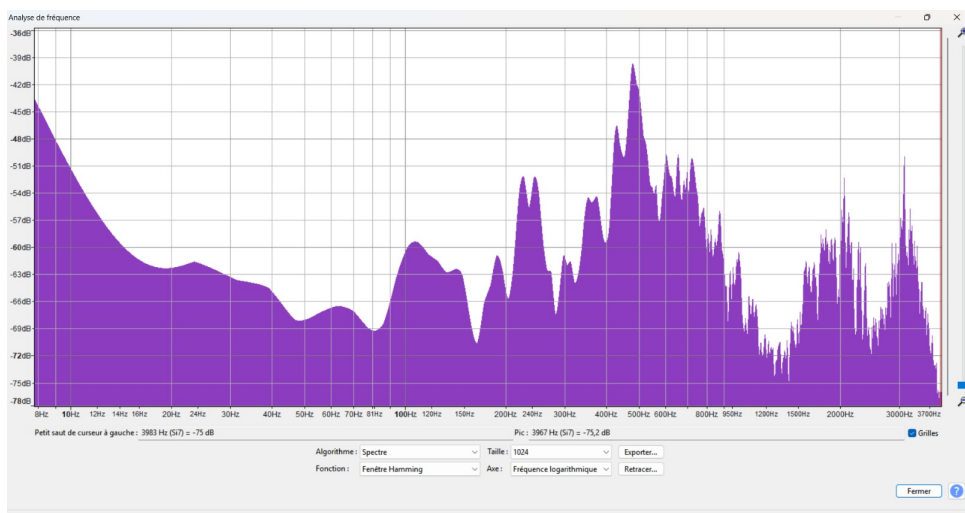


FIGURE 28 – Spectre du signal audio avant application du pré-emphase

On remarque sur le spectre de l'enregistrement avant le pré-emphase que les hautes fréquences n'occupent pas spécialement un grand espace.

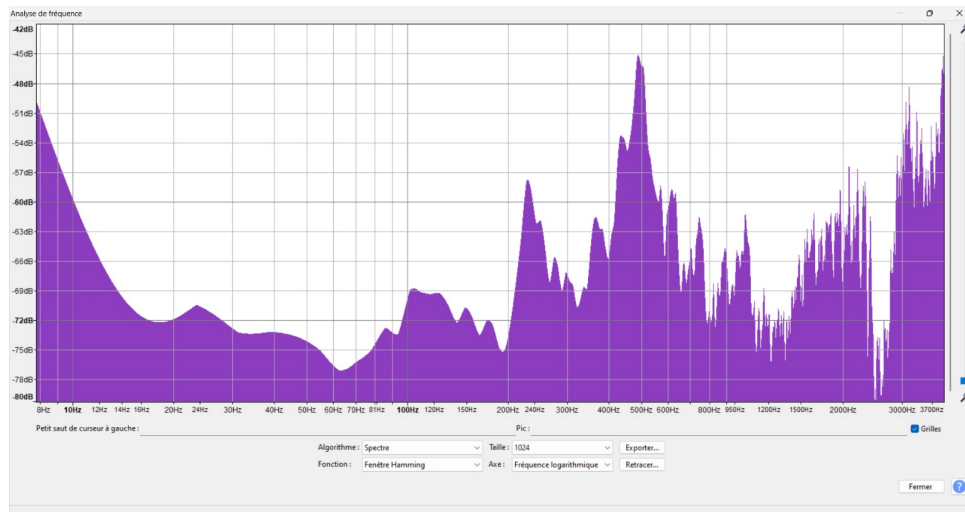


FIGURE 29 – Spectre du signal audio après application du pré-emphase

Après le pré-emphase, on observe immédiatement une augmentation importante des hautes fréquences, ce qui nous permet d'en valider son application.

Cet enregistrement est également écoutable à cette adresse : <https://audio.com/lucie-daix/audio/et6-validation-pre-emphasis>

FFT De la même manière que pour le pré-emphase, on exporte les données de la carte vers un PC afin d'analyser le spectre de l'enregistrement sur Audacity. Pour cette validation, on joue un son de 1 kHz et on observe par la suite le spectre de l'enregistrement :

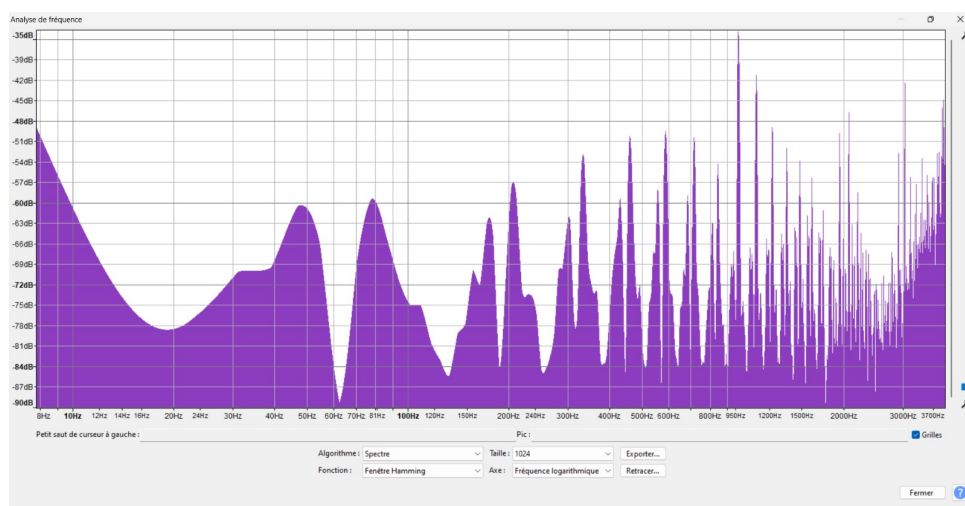


FIGURE 30 – FFT calculée pour une fréquence de 1 kHz

On reconnaît bien ici les pics caractéristiques d'une transformée de Fourier pour un signal de 1 kHz, ce qui nous permet de valider cette partie de l'algorithme de calcul des MFCC.

Coefficients MFC Enfin, on affiche les coefficients calculés juste après la convolution avec le banc de filtre MEL. Ce spectre MFCC a été généré pour un enregistrement du mot « Electronique » :

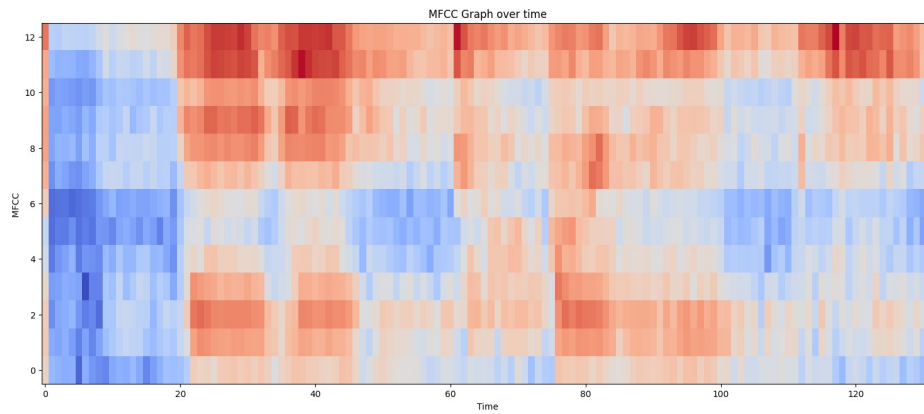


FIGURE 31 – Spectre des MFCC du mot « Electronique »

On remarque bien les différentes inflexions du mot dans les zones de chaleurs importantes du spectre ; successivement : é-le-c[ke]-tro-ni-que. On note également que la première bande du spectre, correspondant en fait à la première frame de l'enregistrement du mot, est coloré de la sorte en raison du son produit par le clic engendré par le relâchement du bouton d'enregistrement.

De même, on analyse le spectre des coefficients MFC après passage de la DCT :

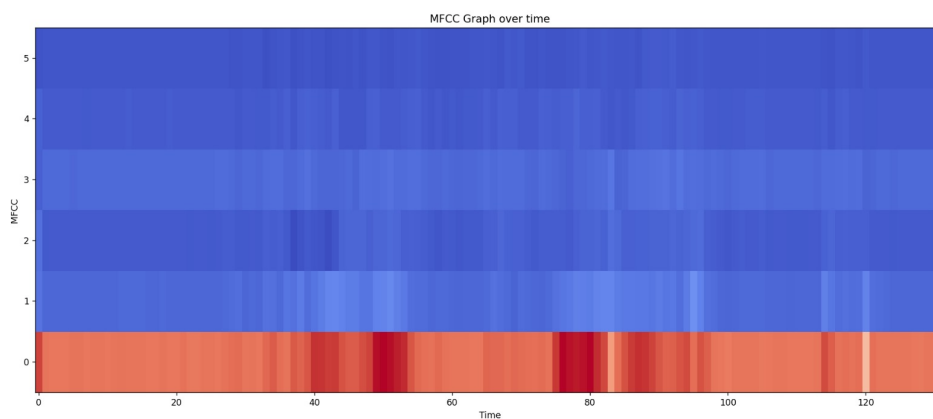


FIGURE 32 – Spectre des MFCC du mot « Electronique » après passage de la DCT

Comme précédemment, on reconnaît les inflexions du mot enregistré, ce qui nous permet finalement de valider le fonctionnement complet de l'algorithme de calcul des MFC et de la DCT.

7.2 Traitement des données

7.2.1 Entraînement du réseau neuronal de test

Pour l'entraînement du réseau neuronal de test contenant les mots « rouge » et « bleu », nous avons relevé toutes les métriques pertinentes nous permettant de valider le bon fonctionnement de celui-ci.

```
22:29:13.774 -> Iteration: 2523 | MSE:0.000100
22:29:13.811 -> Temps d'entrainement : 1553759.00 ms
22:29:13.811 ->
22:29:13.811 -> ==[OUTPUTS]==
22:29:13.811 -> MSE generale : 0.000100
22:29:13.956 -> Rouge | Estimate : 0.9999999
22:29:13.956 -> Rouge | Estimate : 1.0000000
22:29:13.956 -> Rouge | Estimate : 1.0000000
22:29:13.956 -> Rouge | Estimate : 0.9999999
22:29:13.956 -> Rouge | Estimate : 0.9998959
22:29:13.956 -> Bleu | Estimate : 0.0000059
22:29:13.956 -> Bleu | Estimate : 0.0002082
22:29:13.956 -> Bleu | Estimate : 0.0000726
22:29:13.992 -> Bleu | Estimate : 0.0001284
22:29:13.992 -> Bleu | Estimate : 0.0000177
```

FIGURE 33 – Résultats de l'entraînement du réseau neuronal de test

La précision ayant été configurée sur 0.0001, on remarque que le réseau a été entraîné en 2523 itérations ce qui correspond ici à 1553759 ms, soit environ 25 mins ! Malgré un temps d'entraînement extrêmement allongé, on peut tout de même se satisfaire de résultats d'une précision chirurgicale :

	A	B	C	D	E	F
1	Estimations	Erreur	Erreurs ²		MSE	Précision
2	0,9999999	1E-07	1E-14		7,62895E-09	0,99999999
3	1	0	0			
4	1	0	0			
5	0,9999999	1E-07	1E-14			
6	0,9998959	0,0001041	1,08E-08			
7	0,0000059	0,0000059	3,48E-11			
8	0,0002082	0,0002082	4,33E-08			
9	0,0000726	0,0000726	5,27E-09			
10	0,0001284	0,0001284	1,65E-08			
11	0,0000177	0,0000177	3,13E-10			
12						

FIGURE 34 – Calcul de la MSE des données de test pour le réseau neuronal de test

On obtient ici une erreur très largement en dessous de 0.05, ce qui nous permet de valider l'entraînement du réseau neuronal de test.

L'axe d'amélioration principal pour la suite était bien évidemment de réduire le temps d'entraînement, ce qui aurait été possible en baissant la précision requise. Toutefois, nous avons également fait le choix de déléguer cette partie d'entraînement à un PC utilisant la bibliothèque TensorFlow Python en prévision de modèles beaucoup plus complexes contenant nos vraies données de test.

7.2.2 Entraînement du réseau neuronal à deux sorties possibles

Ce réseau neuronal a, comme précisé plus haut, été intégralement entraîné sur PC.

Les données pour son entraînement ont donc été arrangées sous la forme d'un tableau formaté pour la bibliothèque numpy. Les données, des coefficients MFCC après passage de la DCT, se présentent de la sorte :

```
[43.32, 4.07, 2.82, 3.33, 2.14, 1.24],
[37.31, 2.97, 1.69, 3.20, 1.78, 1.23],
[37.39, 2.93, 1.59, 3.08, 1.74, 1.22],
[37.48, 3.03, 1.72, 3.26, 1.78, 1.28],
[37.41, 3.02, 1.76, 3.27, 1.79, 1.26],
[37.42, 3.01, 1.73, 3.16, 1.76, 1.27],
[37.42, 3.04, 1.69, 3.14, 1.67, 1.21],
[37.50, 3.04, 1.79, 3.16, 1.73, 1.25],
[37.45, 3.02, 1.68, 3.16, 1.74, 1.21],
[37.45, 2.95, 1.73, 3.21, 1.77, 1.20],
[37.50, 2.90, 1.64, 3.14, 1.73, 1.24],
[37.43, 3.08, 1.75, 3.19, 1.83, 1.24],
[37.30, 2.99, 1.76, 3.20, 1.68, 1.22],
[37.36, 3.00, 1.68, 3.20, 1.86, 1.28],
[37.55, 3.00, 1.75, 3.20, 1.77, 1.26],
[37.43, 3.01, 1.71, 3.22, 1.80, 1.25],
[37.36, 2.99, 1.70, 3.23, 1.75, 1.22],
[37.57, 3.12, 1.75, 3.23, 1.72, 1.24],
[37.68, 3.00, 1.74, 3.20, 1.78, 1.30],
[37.26, 2.90, 1.72, 3.17, 1.77, 1.21],
[37.51, 3.09, 1.83, 3.28, 1.86, 1.33],
[37.47, 2.99, 1.68, 3.23, 1.82, 1.27],
[37.55, 2.97, 1.66, 3.22, 1.67, 1.21],
[37.32, 3.10, 1.77, 3.19, 1.85, 1.35],
[37.38, 3.03, 1.69, 3.16, 1.67, 1.26],
[37.54, 3.10, 1.71, 3.21, 1.79, 1.23],
[37.28, 2.95, 1.67, 3.20, 1.82, 1.16],
[37.38, 2.84, 1.56, 3.04, 1.64, 1.15],
[37.57, 3.05, 1.70, 3.12, 1.83, 1.29],
[37.52, 3.01, 1.71, 3.09, 1.73, 1.37],
[25.98, 2.73, 1.23, 2.85, 1.24, 0.75],
[37.55, 3.06, 1.78, 3.32, 1.80, 1.16],
[37.53, 3.08, 1.77, 3.22, 1.88, 1.35],
[37.48, 2.96, 1.69, 3.18, 1.77, 1.19],
[38.65, 3.70, 2.00, 2.95, 1.72, 1.14],
[40.14, 4.40, 1.90, 2.95, 1.75, 1.12],
[38.08, 4.93, 1.85, 2.87, 1.31, 1.01],
[38.49, 4.30, 1.23, 2.37, 1.63, 0.64],
[40.61, 4.50, 2.01, 3.09, 2.20, 1.37],
[42.07, 4.45, 2.50, 3.67, 2.08, 1.45],
[34.02, 4.97, 1.68, 2.71, 1.88, 1.08],
[40.47, 4.37, 2.40, 3.34, 1.55, 1.25],
[38.08, 3.89, 2.26, 3.30, 2.14, 1.46],
[39.55, 4.13, 2.01, 3.19, 1.78, 1.24],
[37.96, 3.69, 2.28, 3.59, 2.38, 1.60].
```

FIGURE 35 – Tableau de coefficients utilisé en tant que donnée d’entraînement par le réseau neuronal

Après entraînement du réseau, on obtient les résultats suivants :

```

==[PHASE DE TEST]==
Gui-Cha | Estimate : 0.9985916
Gui-Cha | Estimate : 0.9489998
Gui-Cha | Estimate : 0.9905627
Gui-Cha | Estimate : 0.9981566
Gui-Cha | Estimate : 0.9998587
Thais | Estimate : 0.0018557
Thais | Estimate : 0.0307529
Thais | Estimate : 0.0015571
Thais | Estimate : 0.0000103
Thais | Estimate : 0.0000230

```

FIGURE 36 – Résultats de l'entraînement du réseau neuronal à deux sorties possibles

On remarque que le réseau neuronal a correctement reconnu les 10 mots (5 « Guicha » et 5 « Thaïs »). On calcule alors l'erreur quadratique de ces résultats :

	A	B	C	D	E	F
1	Estimations	Erreur	Erreurs ²		MSE	Précision
2	0,9985916	0,0014084	1,9836E-06		0,00036471	0,99963529
3	0,9489998	0,0510002	0,00260102			
4	0,9905627	0,0094373	8,9063E-05			
5	0,9981566	0,0018434	3,3981E-06			
6	0,9998587	0,0001413	1,9966E-08			
7	0,0018557	0,0018557	3,4436E-06			
8	0,0307529	0,0307529	0,00094574			
9	0,0015571	0,0015571	2,4246E-06			
10	0,0000103	0,0000103	1,0609E-10			
11	0,000023	0,000023	5,29E-10			

FIGURE 37 – Calcul de la MSE des estimations calculées par le réseau neuronal à deux sorties possibles

On remarque que la MSE est largement inférieure à 0.05, ce qui nous permet de valider le fonctionnement de ce réseau neuronal.

7.2.3 Fonctionnalité avancée : entraînement du réseau neuronal à quatre sorties

Enfin, pour l'entraînement du réseau neuronal final à quatre sorties, nous obtenons un résultat d'entraînement exploitable après 5000 cycles en 433.529 secondes. Pour nos tests, nous avons effectué cinq tests pour chaque nom, et avons relevé une MSE inférieure au taux demandé. Cependant, nous remarquons que notre réseau n'est pas très flexible aux perturbations sonores, et possède ainsi des limites d'utilisation dans des cas extrêmes.

	A	B	C	D	E	F
1	Estimations	Erreur	Erreurs ²		MSE	Précision
2	0,9987499	0,0012501	1,5628E-06		0,00201694	0,99798306
3	0,9236643	0,0763357	0,00582714			
4	0,9089175	0,0910825	0,00829602			
5	0,9999891	1,09E-05	1,1881E-10			
6	0,9585832	0,0414168	0,00171535			
7	0,9024175	0,0975825	0,00952234			
8	0,9690177	0,0309823	0,0009599			
9	0,9484832	0,0515168	0,00265398			
10	0,9950735	0,0049265	2,427E-05			
11	0,9907711	0,0092289	8,5173E-05			
12	0,9978263	0,0021737	4,725E-06			
13	0,9442264	0,0557736	0,00311069			
14	0,9999291	7,09E-05	5,0268E-09			
15	0,9884047	0,0115953	0,00013445			
16	0,9841844	0,0158156	0,00025013			
17	0,9641583	0,0358417	0,00128463			
18	0,9921373	0,0078627	6,1822E-05			
19	0,9320188	0,0679812	0,00462144			
20	0,9630622	0,0369378	0,0013644			
21	0,9794881	0,0205119	0,00042074			

FIGURE 38 – Résultats de l'entraînement du réseau neuronal à quatre sorties

Il est donc possible de valider l'entraînement du modèle avec les données de test ci-dessus, où les données vertes correspondent à des Guy-cha, les données jaunes à des Thais, les données rouges à des Lucie et les données bleues à des Toto.

8 Bilan

8.1 État d'avancement

Le projet NeuralSpeech est un projet complètement abouti qui répond à chacune des exigences du cahier des charges.

Dans un premier temps, l'acquisition est une grande réussite.

Tout d'abord, nous avons été capable d'échantillonner le signal à une fréquence de 32 kHz ainsi que de mettre en place un filtre RIF capable d'atténuer d'au moins 30 dB les fréquences supérieures à 4 kHz. Le filtre est appliqué à l'aide d'un buffer circulaire nous permettant de respecter la contrainte du temps de filtrage inférieur à 31 μ s.

Ensuite, l'aboutissement du projet repose aussi sur la restitution de manière claire de l'enregistrement du mot « Electronique » ainsi que de la séparation en frames de 256 échantillons qui vérifie le bon fonctionnement du recouvrement de chaque frame pour éviter la perte d'information.

Enfin, nous avons su générer 13 MFCCs à partir d'un enregistrement d'une seconde.

Dans un second temps, le traitement des données répond en tous points à nos attentes.

Notre compréhension des réseaux de neurones convolutifs s'est bien développée à l'aide des données de test et nous a permis d'établir une architecture robuste pour le réseau de neurones du projet, comme le montre la MSE des données de test (pour les mots « bleu » et « rouge ») qui est bien inférieure à 0,05.

De plus, comme le prévoit le cahier des charges, notre jeu de données d'entraînement doit contenir au moins 100 éléments. La bonne gestion des enregistrements et la mise en place d'un tableau de données intelligemment codé ont été les clés de la réussite.

Aussi, un autre accomplissement qui révèle un projet prometteur, c'est la capacité de notre réseau de neurones à pouvoir classer nos 20 mots de tests en 4 catégories distinctes, et cela avec moins de 5% d'erreur.

Pour conclure sur les aboutissements de notre projet, compte tenu que les deux parties mentionnées en amont ont été parfaitement traitées et que la solution développée répond aux spécifications techniques du cahier des charges, nous avons pu transformer ce projet en un jeu interactif et fonctionnel.

En effet, NeuralSpeech est un projet de reconnaissance vocale basé sur une IA embarquée.. Cependant nous avons voulu rendre ce projet attractif et amusant pour l'utilisateur et ainsi faire évoluer NeuralSpeech en tant que jeu interactif. C'est pourquoi nous avons décidé de créer le jeu : *Qui De Nous 4 ?*. Le but est de répondre oralement aux 30 questions posées en indiquant qui parmi les 4 prénoms proposés correspond le mieux à la situation décrite. Le jeu se déroule en 4 manches. Le gagnant est celui qui récoltera le plus de bonnes réponses après de son passage.



FIGURE 39 – Interface processing de Qui De Nous 4 ?

8.2 Pertinence de la solution technique

Quand bien même le projet est une grande réussite, nous avons tout de même rencontré certaines difficultés.

La difficulté la plus marquante a été la lecture des données sur Audacity. En effet, à cause de l'utilisation d'un mauvais interpréteur de commandes (Powershell au lieu de CMD), les données transmises par *plink* (la version CLI de PuTTY) étaient erronées et faussaient tous les enregistrements.

De plus, les enregistrements des données d'entraînements du modèle à 4 sorties nous ont posé problème. En effet, certains de ces enregistrements ont été effectués dans des conditions sonores différentes de celles des premières données. Cela a créé un déséquilibre dans le modèle, qui a pu être corrigé en ré-enregistrant, cette fois convenablement, l'ensemble des données d'entraînement. Néanmoins, nous avons recensé plusieurs perspectives d'évolutions.

Dans un premier temps, utiliser un microphone avec une meilleure isolation du bruit ou des technologies de réduction de bruit intégrées comme le son les casques audios pour gamers. Dans cette idée, nous pouvons penser au microphone MEMS qui reste accessible pour ce type de projet.

Puis dans un second temps, un autre axe d'amélioration peut être lié à l'entraînement de notre réseau de neurone. Il serait judicieux d'entraîner notre réseau avec des jeux de données plus diversifiés. Par exemple utiliser des données provenant de différents environnements sonores normalisés pour rendre le modèle plus adaptable.

8.3 Bilan sur le travail d'équipe

- Lucie : J'ai développé de grandes capacités dans l'acquisition de signal. L'échantillonnage, qui me semblait être une notion encore abstraite, est désormais bien maîtrisé. De plus, la prise en main rapide de logiciels qui m'étaient étrangers, tels qu'Audacity, me conforte dans mon profil d'ingénieur, capable de s'adapter rapidement à tout type de situation. Cette faculté précieuse sera un atout majeur pour ma prochaine recherche de stage.
- Thaïs : J'ai approfondi mes connaissances sur les filtres et les algorithmes MFCC. Ma curiosité naturelle m'a poussé à explorer ces concepts, et le travail de groupe m'a permis de développer des compétences en collaboration et entraide. Ces compétences seront essentielles pour ma prochaine recherche de stage, où la capacité à travailler en équipe et à résoudre des problèmes techniques sera valorisée.

- Guy-Charbel : En tant que chef de projet, j'ai supervisé et coordonné l'ensemble du projet. Cette expérience m'a permis de développer des compétences en gestion de projet, en technique et en prise de décision. Ces compétences seront mises en avant lors de ma prochaine recherche de stage, démontrant ma capacité à diriger des équipes et à mener des projets à bien.
- Thomas : J'ai acquis des compétences solides en réseaux de neurones et en utilisation de bibliothèques de réseaux neuronaux. La patience et la persévérance ont été cruciales pour surmonter les défis techniques rencontrés. Ces compétences seront mises en avant sur mon CV, montrant ma capacité à gérer des projets complexes et à persévérer face aux obstacles.

L'équipe s'est divisée en deux groupes : Thaïs et Lucie se sont concentrées sur l'acquisition du signal, tandis que Thomas s'est occupé du traitement des données. Guy-Charbel a aidé les deux groupes lorsque des points bloquaient, et le projet a été terminé dans les temps, avec une livraison prévue dimanche et rendue le jour espéré. La communication au sein de l'équipe a été bonne. Cependant, il a été difficile de planifier les tâches longtemps à l'avance en raison d'un manque de vision claire, ce qui est un point à retravailler.

9 Sources

Documents utilisés et sites internet consultés pour développer le projet.

Différence filtre RIF et RII :

<https://www.advsolned.com/difference-entre-les-filtres-rii-et-rif-un-guide-pratique-de-conception/>